

CANDEX HANDBOOK

The complete handbook

Every Candex handbook in one document — the product, the domain, the architecture, and how the app, the API, and the dashboard are built, tested, and shipped.

Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app



CANDEX HANDBOOK

Product

What Candex is as a product — the pitch, who it serves, the feature set by status, and where it's headed.

Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

Product

This handbook is the product view: the vision, the audience, the feature set, and the roadmap. For a hands-on tour of what the app does **today**, see Understanding Candex. Forward-looking items here are tagged by status, so the picture stays honest.

The pitch

Candex turns a scattered pile of beverage cans into a living, organized collection you carry in your pocket. Discover new drinks, track the cans you own, and see where each one ranks in a world-wide dex shared by every collector.

Who it’s for

- **Collectors** — people who keep and grow a real collection of cans and want it catalogued properly.
- **Casual fans** — drinkers who want to remember what they liked and find more of it.
- **The curious** — anyone who enjoys browsing drinks, brands, and the details behind them.

Feature set

Status legend: **Live** (shipped) • **In progress** (being built) • **Planned** (on the roadmap) • **Exploring** (product direction, not started).

Feature	What it does	Status
Personal Dex	Unlock drinks and build your own collection	Live
Search & discover	Browse the catalogue, open a drink for its details	Live
World Dex	A global ranked list, with position history	Live
Drink details	Names, brand, flavours, composition and claims per country	Live
Themes & language	Light / Dark / System, English and French	Live
Security & backup	Firebase sign-in, and a backed-up collection	Live
Scan a can	Add a can from a photo via image recognition	In progress
Statistics	Collection stats — counts, top brands, habits	Planned
Community	Share finds, tips, and events with other collectors	Exploring

The *Scan a can* flow is partly in place on the API, which already does image and colour analysis; the end-to-end scanning experience is still being built *(to confirm on the app side)*. *Statistics* and *Community* reflect the product vision and are not shipped.

Roadmap

Near-term items from the product board:

- **Drink editing** — edit an existing drink.
- **Drink deletion** — remove a drink.
- **Cans page** — a dedicated view for a drink's cans.
- **Reordering drinks** — move a drink's position (`dexPosition`).
- **API CI/CD** — finish the delivery pipeline for the API.

Product principles

- **Solo-built, end to end** — one engineer owns the app, the dashboard, and the API; the handbooks exist so that doesn't become a bottleneck.
- **Bilingual by default** — every screen ships in English and French.
- **Yours, your way** — themes and language are the user's choice, and their collection is backed up so it is never lost.

Where to go next

- Using the app today → Understanding Candex
- How it looks and feels → UI / UX
- How it's built → Architecture



CANDEX HANDBOOK

Understanding Candex

Understand Candex as a product — what it is, how it feels to use, and the ideas behind it.

Written by
Eliott Barbet

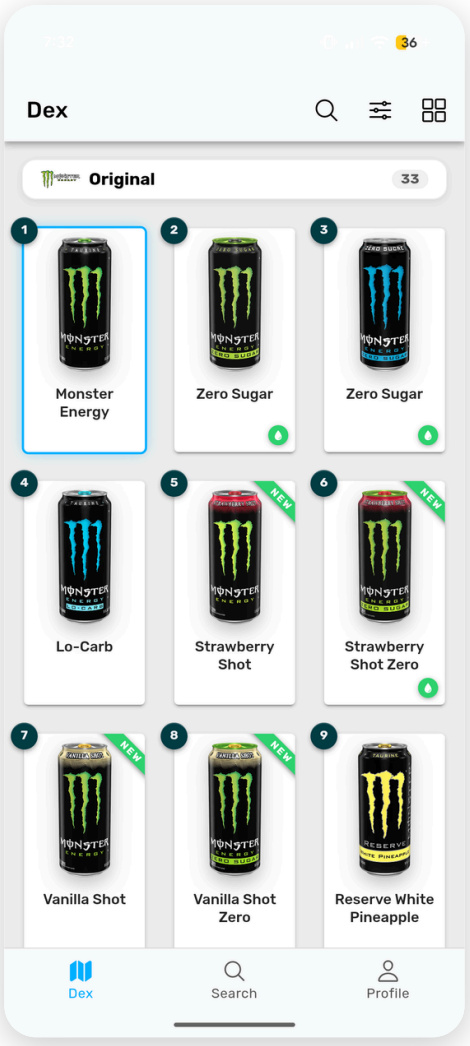
Version 0.23.0
November 2026
candex.app

Understanding Candex

You are travelling, and on a shop shelf you spot an energy drink you have never seen before. Have you tried this one? Which editions exist? Candex turns that small moment into a game.

What Candex is

Candex is a mobile app that turns **discovering and collecting drinks into a game**. You find beverages from around the world, mark the ones you have tasted, and fill in a personal catalogue — your **World Dex** — the way you would complete a collection in a video game.



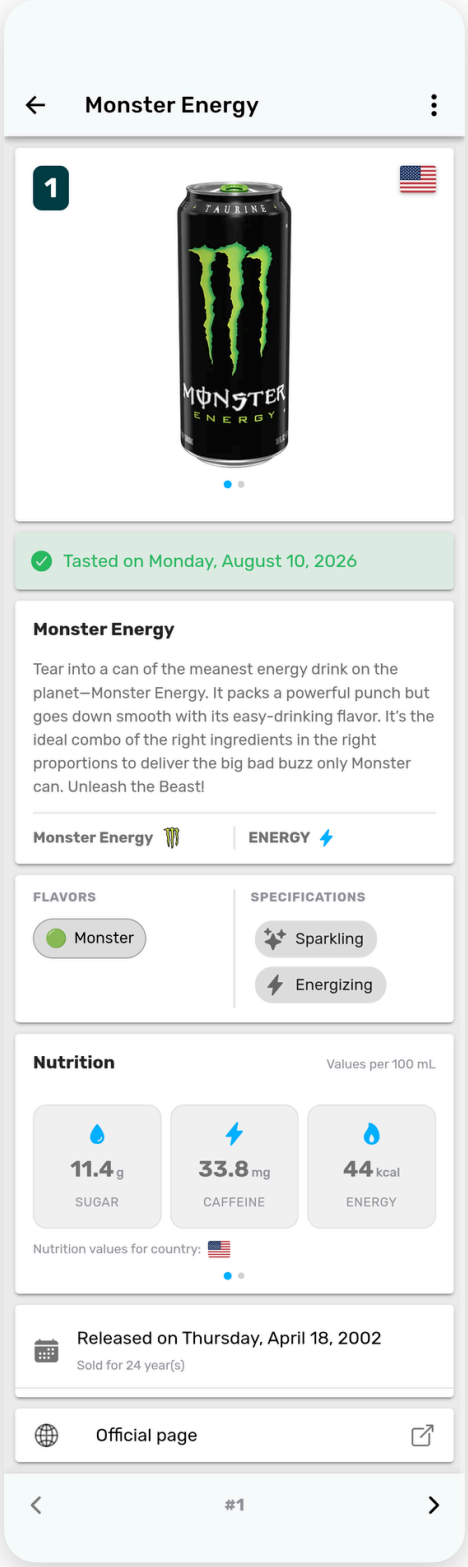
Your World Dex — each drink, filled in as you taste it.

The whole app is one simple loop, three moves:

1. **Discover** a drink — browse the catalogue by brand, series, and flavour.
2. **Taste it** — mark a drink as tasted; it lights up in your Dex, with the date you had it.
3. **Progress** — watch your World Dex fill in, and see what you are still missing.

What a drink looks like

Every drink has its own page: the can, where and when you tasted it, a description, its brand and category, its flavours, and nutrition per 100 mL.



A drink's page — tasted, with flavour, specs, and nutrition.

Core concepts

A few words show up throughout Candex. Here they are in plain terms; the planned **Glossary / Domain** handbook defines each one precisely.

Term	What it means
Drink	A beverage in the catalogue — the thing you collect.
Can	A physical can of a drink; one drink can have several (editions, countries).
Brand	The company that makes a drink — Monster, Coca-Cola.
Series	A line within a brand — for example Monster's *Original* line.
Flavour	A drink's flavour, shown per language.
World Dex	Your collection catalogue — what you are filling in.
Taste	Marking a drink as tasted, so it counts in your Dex (also called *unlock*).

> **Note**
>
> Some older product notes mention ideas like *intake* or *wishlist*. These are not part of the app today, so they are left out here and will be documented if they are built.

Who it is for

- **Explorers** — who want to discover local, rare, or popular drinks from elsewhere.
- **Collectors** — who want to complete their World Dex and chase what they are missing.
- **The curious** — who just want a little extra info about a drink in their hand.

Where to go next

- **Here to work on the code?** → Working on Candex
- Want the exact definitions of each term? → Glossary / Domain *(planned)*
- Curious about the product vision and roadmap? → Product *(planned)*

Your first step: the fastest way to understand Candex is to use it — open the app, find a drink you have already had, and mark it as tasted. That is your first entry in the World Dex.



CANDEX HANDBOOK

Working on Candex

Get your bearings in the Candex codebase — the system, the repositories, the stack, and your first step.

Written by
Eliott Barbet

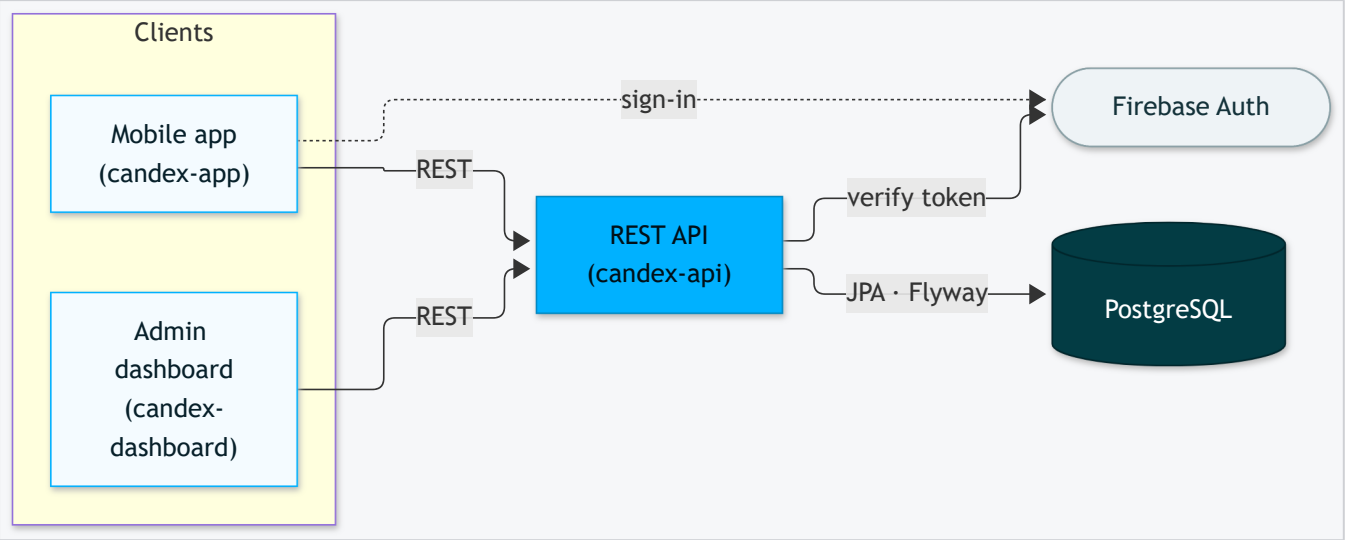
Version 0.23.0
November 2026
candex.app

Working on Candex

This page orients you if you are going to work on Candex. For the product itself — what it is and how it feels to use — start with Understanding Candex.

The system at a glance

Candex is not one app but a few pieces working together. Users touch the **mobile app**. An **admin dashboard** keeps the drink catalogue accurate. Both talk to a single **REST API**, which holds the domain and stores everything in **PostgreSQL**. Sign-in is handled by **Firebase**.



Repositories

Repository	Role	Status
`candex-app`	Mobile client (Angular + Ionic + Capacitor)	Active
`candex-dashboard`	Admin dashboard for managing the catalogue	Active
`candex-api`	REST API and domain model, backed by PostgreSQL	Active
`candex-handbook`	This documentation site (Astro Starlight)	Active
`candex-crawler`	Crawler that gathers drink and catalogue data	Active
`candex-ai`	AI module (Python)	Early / placeholder
`candex-backups`	Firestore backup tooling	Deprecated — predates the move to PostgreSQL
`candex-api-legacy`	Previous API (Node.js / Express)	Deprecated

> **Note**
>
> `candex-crawler` lives in its own repository (outside the main workspace); its role is summarised here from its purpose and will be detailed in its own handbook.

Tech stack

Mobile app — `candex-app`

Angular v20 | **Ionic** v8 | **Capacitor** v7

TypeScript

NgRx
Store • Effects • Entity

RxJS

Firebase
Auth

Admin dashboard — `candex-dashboard`

Angular
v20

Ionic
v8

TypeScript

Angular Material

Firebase
Auth

Back-end — `candex-api`

Spring Boot
v3.5

Java
21

Maven
build (wrapper)

PostgreSQL

Flyway
migrations

MapStruct
mapping

Spring Security
OAuth2

OpenAPI
springdoc

Quality & delivery

Docker
Compose

JUnit
+ Mockito

Testcontainers

JaCoCo
coverage

ESLint

Prettier

Cypress
e2e

Where things live

- **Domain & API** — `candex-api`. Entities and business rules in `src/main/java`; schema in PostgreSQL, evolved through Flyway migrations under `src/main/resources/db/migration`.
- **Mobile UI** — `candex-app` (Angular + Ionic, NgRx for state).
- **Admin UI** — `candex-dashboard` (Angular + Ionic + Angular Material).

The codebase is the source of truth: when documentation and code disagree, the code wins and the docs are corrected.

Running it locally

Candex runs as three apps — the mobile app and the dashboard (Ionic/Angular, run with npm) and the API (Maven + Spring Boot, with PostgreSQL), each in three environments (local, dev, prod). The full setup — prerequisites, cloning, environment files, the database, and the per-app commands — lives in the **Local Installation** handbook.

Where to go next

- Set up your machine → Local Installation
- Understand how the pieces fit, and why → Architecture & Overview
- The precise domain language → Glossary / Domain *(planned)*

Your first step: clone the repositories, then follow Local Installation to get the API and at least one front-end running against a local database.



CANDEX HANDBOOK

Local Installation

Get Candex running on your machine — the API with a local database, the mobile app, and the admin dashboard.

Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

Local Installation

This guide gets Candex running on your machine: the **API** with a local PostgreSQL database, the **mobile app**, and the **admin dashboard**. If you have not yet, skim Working on Candex first for the big picture.

The local database is built **from scratch** by the API's Flyway migrations (which include seed data), so you do **not** need access to any server.

What you will have running

Service	URL
REST API	`http://localhost:8080` (Swagger at `/swagger-ui.html`)
Mobile app	`http://localhost:8100`
Admin dashboard	`http://localhost:8101`
PostgreSQL	`localhost:5432`

Prerequisites

- **Git**
- **Node.js 20+** and **npm 10+** — for the app and dashboard
- **JDK 21** — for the API (a Maven wrapper, `mvnw`, is included, so a separate Maven install is optional)
- **Docker** and Docker Compose — for the local Postgres + API stack
- A **Firebase project**'s config — for the app's authentication (see the app section)
- **Android Studio** — optional, only for native Android builds

Get the code

Clone the repositories into one folder:

```
git clone https://github.com/RusterX16/candex-api
git clone https://github.com/RusterX16/candex-app
git clone https://github.com/RusterX16/candex-dashboard
```

1 — API and database (`candex-api`)

Create the environment file from the template and fill in values:

```
cd candex-api
cp .env.example .env     # then edit .env
```

The recommended path is the helper script, which runs a throwaway containerised stack (Postgres 17 + API) from `compose.local.yaml`:

```
./scripts/candex.sh up        # start local Postgres + API
```

On startup the API runs its **Flyway migrations**, which create the schema and seed baseline data (brands, app versions, and more) — a working database with no server access required.

Other useful commands:

```
./scripts/candex.sh down      # stop the stack (data kept)
./scripts/candex.sh reset     # wipe the local database volume
./scripts/candex.sh psql      # open psql on the local database
./scripts/candex.sh logs      # follow the API logs
```

The API is then at `http://localhost:8080`, with Swagger UI at `/swagger-ui.html`.

> **Without Docker**
>
> If you prefer to run your own PostgreSQL, set `CANDEX_DB_URL`, `CANDEX_DB_USERNAME`, and `CANDEX_DB_PASSWORD`, then start the API with `./run-dev.sh` (or `./mvnw spring-boot:run -Dspring-boot.run.profiles=dev`).

2 — Mobile app (`candex-app`)

Install dependencies:

```
cd candex-app
npm install
```

The app generates its Angular environment from env files (`scripts/generate-env.js`). Create them from the template:

- `.env.dev` and `.env.prod` — one Firebase project each, plus `API_URL` (copy `.env.example`)
- `.env.local` — just `API_URL=http://localhost:8080`, which overrides the API URL for local runs

Firebase config. The app authenticates through Firebase, so `.env.dev` / `.env.prod` need the seven `FIREBASE_*` values, and the Android build needs `android/app/google-services.json`. Get these from your Firebase project (Project settings → your web app) or from the project owner. No Firebase values are included in this documentation.

Run against your local API:

```
npm run start:local     # http://localhost:8100 (dev Firebase + localhost API)
```

`start:local` pairs with the API's **dev** profile — keep both on `dev`. For a native Android build (needs Android Studio): `npm run android:dev`.

3 — Admin dashboard (`candex-dashboard`)

```
cd candex-dashboard
npm install
cp .env.example .env     # set API_URL_LOCAL and the admin credentials
npm start                # http://localhost:8101
```

Putting it together

For a consistent local setup, keep the app's **local** mode and the API's **dev** profile together. Typical run order: start the API (`candex.sh up`), then the dashboard (`npm start`) and the app (`npm run start:local`).

Troubleshooting

- **Port already in use** — stop the process holding `8080`, `8100`, `8101`, or `5432`, or change the port.
- **API cannot reach the database** — make sure Docker is running and `candex.sh up` reports a healthy `db`; check the `DEV_DB_*` values in `.env`.
- **App build fails with "Missing env vars"** — `generate-env.js` needs `API_URL` plus all seven `FIREBASE_*` values in the loaded `.env.*` file.
- **Empty database** — migrations run when the API starts; check `candex.sh logs` to confirm Flyway applied them.

Where to go next

- API internals → Back-end *(planned)*
- Environments and releases → Deployment *(planned)*



CANDEX HANDBOOK

Architecture & Overview

How Candex is put together — the domain model, the API's layers, authentication, data, the front-ends, and deployment. The map; each area has its own handbook.

Written by
Eliott Barbet

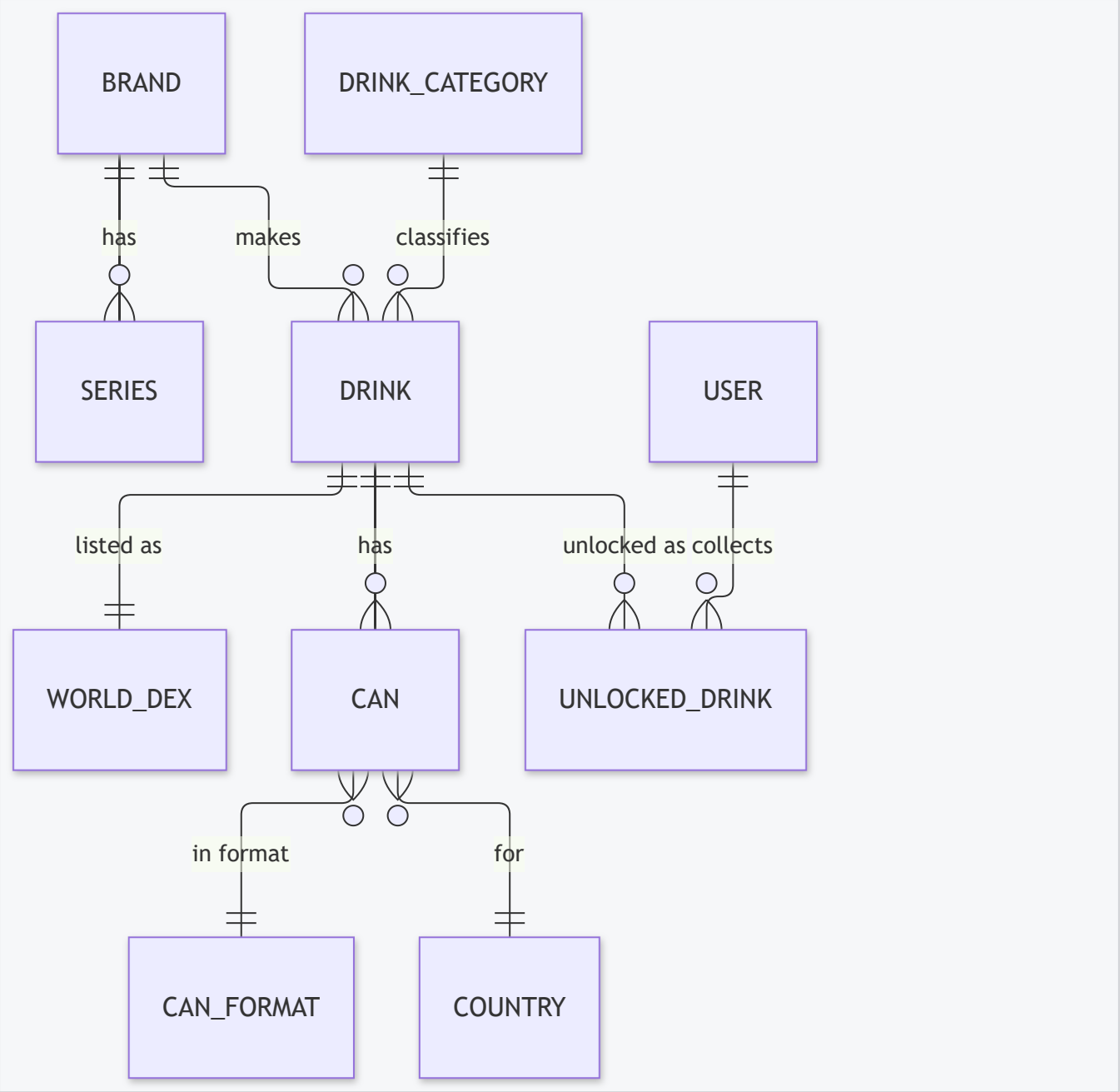
Version 0.23.0
November 2026
candex.app

Architecture & Overview

This is the map of how Candex is built. It stays high-level and links out: the **Back-end**, **Front-end**, **Database**, and **Deployment** handbooks go deep on each area. For the moving parts and repositories, see Working on Candex.

The domain model

The catalogue is a graph of entities around **Drink**. A drink belongs to a **brand** and a **category**, is listed once in the **World Dex**, and has one or more **cans** (per country, format, and colours). Users build their collection by **unlocking** drinks.



Around this core sit supporting entities — `Flavor` (with categories and translations), `Color`, `Language`, `DrinkLocalization`, and `WorldDexArrival` / `WorldDexHistory` for ordering. Catalogue entities (`Brand`, `Series`, `Flavor`, `CanFormat`) carry an **`introduced_in_version`** link to `AppVersion`, so the catalogue is versioned alongside the app. The full entity model is in the **Database & Model** handbook.

API structure

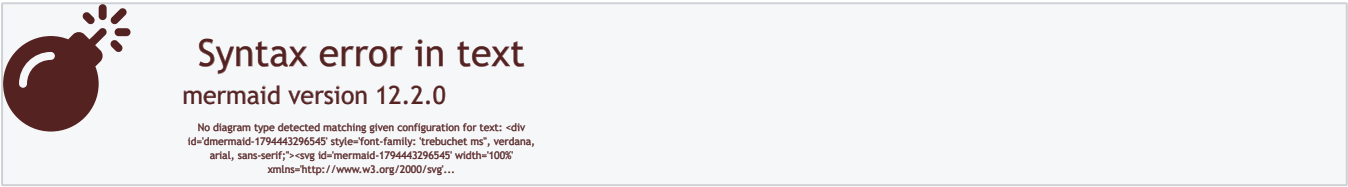
`candex-api` is a layered Spring Boot application under `dev.ruster.candex`:

Layer	Package	Responsibility
Web	`controller`	REST endpoints
Contract	`dto` (`request` / `response`), `mapper`	API shapes, mapped to/from entities with MapStruct
Business	`service` (`core`, `analysis`)	Domain logic
Data	`repository`, `entity`	Spring Data JPA over PostgreSQL
Cross-cutting	`security`, `exception`, `validation`, `config`, `job`	Auth, errors, validation, config, scheduled jobs

The `dto` + `mapper` boundary keeps the API contract decoupled from the JPA model — controllers never expose entities directly.

Authentication & security

The app signs in through **Firebase**; the API verifies the Firebase ID token on every request (`FirebaseAuthenticationFilter` / `FirebaseAuthenticationService`, backed by the Firebase Admin SDK) and authorises by role (`ROLE_ADMIN`, `ROLE_USER`).



Data & migrations

The domain lives in **PostgreSQL**. The schema is owned by **Flyway** migrations under `candex-api/src/main/resources/db/migration`: every change is a versioned `V__*.sql` file, and seed data (brands, app versions, and more) ships as migrations too — which is why a fresh database comes up usable. A read-only SQL-console role is provisioned by migration for safe inspection.

The front-ends

Both front-ends are Angular + Ionic and talk to the same REST API.

- **Mobile app** (`candex-app`) — state managed with **NgRx**, organised into feature slices: `auth`, `dex`, `drink`, `user`, `preferences`, `app`.
- **Admin dashboard** (`candex-dashboard`) — Angular + **Angular Material**, used to keep the catalogue accurate.

See the **Front-end** handbook for structure and conventions.

Deployment topology

Each environment is a Docker Compose stack of an **API** container and a **PostgreSQL 17** container:

Environment	Compose file	Notes
local	`compose.local.yaml`	Throwaway stack via `scripts/candex.sh` (see Local Installation)
dev	`compose.dev.yaml`	`api-dev` + `db-dev`; API on host `8081`, debug `5005`
prod	`compose.prod.yaml`	`api-prod` + `db-prod`

> **To confirm**
>
> TLS and reverse proxying are not in these compose files, though a `CADDY_EMAIL` variable exists — so the public entrypoint (likely **Caddy**) appears to be handled at the server level. The **Deployment** handbook documents the exact topology.

Design notes

Grounded from the code and config; items marked *to confirm* need your input.

- **Layered API with a DTO/mapper boundary** — the `dto` + `mapper` packages keep the public contract separate from the JPA entities, so the database model can evolve without breaking clients.

- **Firebase for identity, PostgreSQL for the domain** — authentication is delegated to Firebase while all catalogue and collection data is relational in Postgres (evident from the dependencies and config).
- **Flyway-owned schema** — the database is never edited by hand; every change is a versioned migration, which also makes local setup reproducible.
- **Separate admin dashboard** — catalogue management is a distinct app from the consumer mobile app. *(Rationale to confirm.)*
- **Versioned catalogue** — catalogue entities reference `AppVersion`. *(Intended use to confirm.)*

Where to go next

- The data model → Database
- The API layers → Back-end
- The front-end → Front-end



CANDEX HANDBOOK

Database & Model

The Candex data model — schema organisation, the full entity graph, reference data, Flyway migrations, World Dex ordering, and the read-only SQL console.

Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

Database & Model

This is the reference for Candex's data. The domain lives in **PostgreSQL**, and the schema is owned entirely by **Flyway** migrations. For the big picture and the simplified core, see Architecture & Overview.

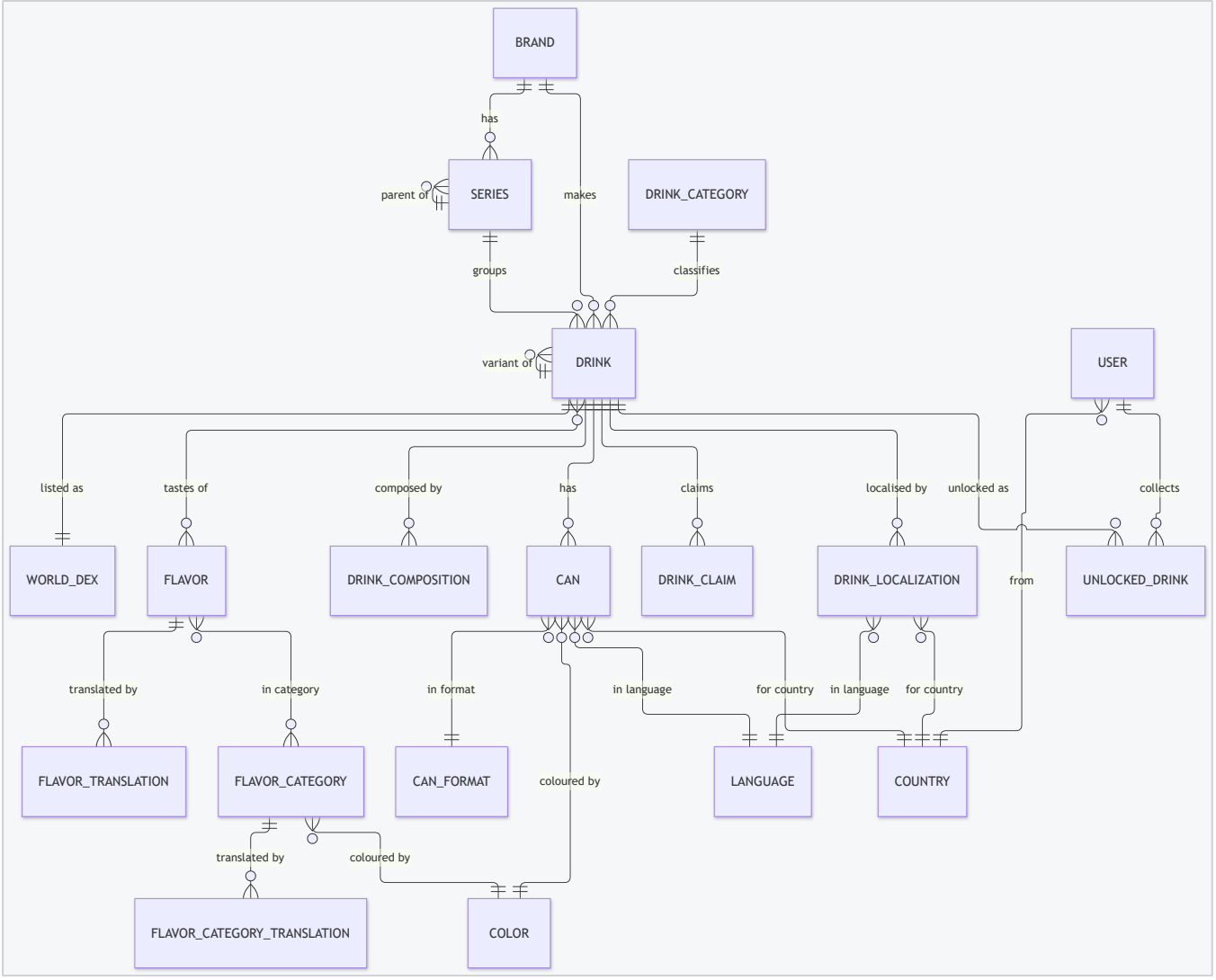
Schema organisation

The baseline schema is laid down by migrations `V1`–`V6`, in three areas plus the runtime machinery:

Migration	Area
`V1` — core reference schema	Reference tables: countries, languages, colours, categories, app versions
`V2` — user data schema	Users and their unlocked drinks
`V3` — catalogue & dex schema	Brands, series, drinks, cans, flavours, World Dex
`V4` — runtime routines	PostgreSQL functions
`V5` — runtime triggers	Triggers that keep the World Dex ordered
`V6` — static reference data	Seeds the reference tables

The entity model

The catalogue is a graph around **Drink**. A drink belongs to a brand, a series, and a category; has cans, localisations, compositions, and claims; tastes of one or more flavours; and is listed once in the World Dex. Users collect drinks by unlocking them.



A few points the diagram abstracts:

- **Versioned catalogue.** Most catalogue and reference entities (`Brand`, `Series`, `Flavor`, `Color`, `Country`, `Language`, `DrinkCategory`, `CanFormat`, `User`) carry an `introduced_in_version` link to `AppVersion`, so every row is tied to the app version that introduced it. These edges are left out of the diagram to keep it readable.
- **Composite-key join tables.** `UserUnlockedDrink` (user + drink), `DrinkLocalization` (drink + country + language), `FlavorTranslation`, `FlavorCategoryTranslation`, and `DrinkCountry` use composite primary keys (see `entity/id`).
- **World Dex ordering.** `WorldDexArrival` and `WorldDexHistory` track the order and arrival of entries; that order is maintained in the database (see below).

Reference & seed data

Reference tables (countries, languages, colours, drink categories, can formats) and the first app versions are seeded by migrations, so a fresh database is immediately usable. Brand and series baselines (for example Monster and Coca-Cola) are seeded in later migrations (`V13`, `V14`). Because the catalogue is versioned, reference and catalogue rows are introduced against a specific `AppVersion`.

Migrations & Flyway

Flyway owns the schema (`spring.flyway.enabled=true`, `locations=classpath:db/migration`).

- `V1`–`V6` are the **canonical baseline** for empty databases; existing non-empty databases are baselined at version `6`.
- Every later change is an **incremental** `V7+` migration.
- Applied migrations are **immutable** — never rename, reorder, or edit a migration after it has been deployed.
- Legacy snapshot scripts are kept under `src/main/resources/sql/archive/`.

To change the schema or reference data, add the next `V<n>__short_description.sql` file; it runs automatically the next time the API starts. A real one, `V28__add_app_version_is_mandatory.sql` — a header saying *why*, then the smallest change that does it:

```
--
═══════════════════════════════════════════════════════════════════════════
-- Forced app update (feature/262)
-- ═══════════════════════════════════════════════════════════════════════════
-- A mandatory version is the minimum the app must run: an installed version
-- below the highest mandatory one is blocked behind a "Update required" screen.
ALTER TABLE app_versions
    ADD COLUMN is_mandatory BOOLEAN NOT NULL DEFAULT FALSE;
```

Note the `DEFAULT`: a `NOT NULL` column added to a table that already has rows must say what those rows get, or the migration fails in production while passing on an empty local database.

World Dex ordering

The World Dex is an **ordered** catalogue, and that order is kept consistent **in the database** rather than in the app. PostgreSQL routines (`V4`) and triggers (`V5`) shift entries when drinks are inserted or deleted, so positions stay contiguous and correct. This ordering logic has been refined across several migrations (`V9`, `V12`, `V17`–`V20`); the migration files are the source of truth for the exact behaviour. The **Back-end** handbook covers how the API exposes this.

Read-only SQL console

Migration `V16` provisions a **read-only database role** for safe inspection. Its password comes from the Flyway placeholder `readonly_password` (`CANDEX_SQLCONSOLE_READONLY_PASSWORD`). This role backs the admin dashboard's SQL console, so queries can be run against the database without write access.

Conventions

- Tables and columns are `snake_case`; foreign keys are `<entity>_id` (or a natural code such as `country_code`, `language_code`).
- Join and translation tables use **composite primary keys**, defined in `entity/id`.
- Migrations are **versioned and immutable**; the database is never edited by hand.
- Catalogue and reference rows are tied to an `AppVersion` via `introduced_in_version`.

Where to go next

- The layers that use it → Back-end
- The terms behind the tables → Glossary
- How it's backed up → Backups



CANDEX HANDBOOK

Back-end

The Candex REST API — request flow, the response envelope, the API surface, services, error handling, authorization, scheduled jobs, and configuration.

Written by
Eliott Barbet

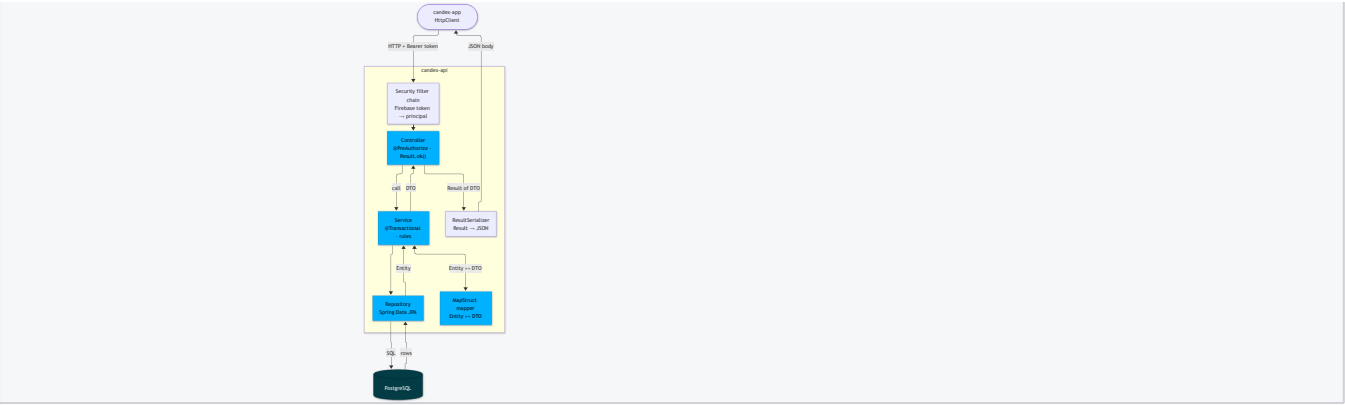
Version 0.23.0
November 2026
candex.app

Back-end

`candex-api` is a layered Spring Boot application. This handbook covers how it is built; for the domain it serves, see Database & Model, and for the system around it, Architecture & Overview.

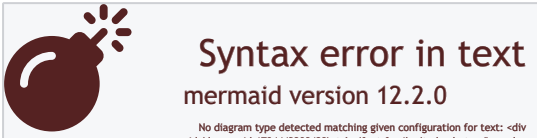
Request flow

Every request flows through the same layers. Controllers return a typed **`Result`** envelope; services hold the logic; repositories are Spring Data JPA; and the boundary between the JPA entities and the API shapes is crossed by **MapStruct** mappers.



A request, end to end

`GET /brands/{id}` as it really runs — `BrandController` → `BrandService` → `BrandRepository` → `BrandMapper`. Every read follows this shape; writes run it in reverse (DTO → `toEntity` → `save`).



Three things to notice:

- **The entity never leaves the service.** The mapper runs inside the service, so the controller only ever sees DTOs.
- **Exceptions are caught in the controller**, not in a global `@ControllerAdvice`: each endpoint lists the `CustomExceptions` it expects and returns `e.getResponse()`. An exception it does not catch falls through to Spring's default error handling.
- **Transactions live on the service**, on the write paths (`@Transactional` on `createUser`, `DrinkService` imports, …). Plain reads like this one have none: lazy relations are resolved by the mapper thanks to Spring's default *open-in-view*, which keeps the JPA session open for the whole request.

The same request in code

The three classes behind that request, trimmed of their Swagger annotations. This is the template every resource follows.

The **controller** declares the route and the access rule, delegates, and turns the outcome into a response — nothing else:

```
@RestController
@RequiredArgsConstructor
@RequestMapping(ApiPaths.BRANDS)
@PreAuthorize(SecurityExpressions.USER_OR_ADMIN)
public class BrandController {

    private final BrandService brandService;

    @GetMapping("/{id}")
    public ResponseEntity<Result<Brand>> getBrandById(@PathVariable int id) {
        try {
            return Result.ok(brandService.getBrandById(id));
        } catch (ElementNotFoundException e) {
            return e.getResponse();
        }
    }
}
```

The **service** reads through the repository, raises a typed exception when a rule fails, and hands back a DTO — the entity never escapes:

```
@Service
@RequiredArgsConstructor
public class BrandService {

    private final BrandRepository brandRepository;
    private final BrandMapper brandMapper;

    public Brand getBrandById(int id) {
        return brandMapper.toDTO(brandRepository.findById(id).orElseThrow(
                () -> new ElementNotFoundException(BrandEntity.class, "id", id)
        ));
    }
}
```

The **mapper** is an interface: MapStruct generates the implementation at compile time. Fields with the same name map themselves; only the differences are declared:

```
@Mapper(componentModel = "spring")
public interface BrandMapper {

    @Mapping(source = "appVersion.version", target = "introducedInVersion")
    Brand toDTO(BrandEntity entity);

    List<Brand> toDTO(List<BrandEntity> entityList);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "createdAt", ignore = true)
    @Mapping(target = "updatedAt", ignore = true)
    @Mapping(source = "introducedInVersion", target = "appVersion.version")
    BrandEntity toEntity(Brand dto);
}
```

Each layer has exactly one job:

Layer	Responsibility	Must not
Controller	Map HTTP to a method, enforce access (`@PreAuthorize`), return a `Result`. Thin.	Hold business logic or touch the database
Service (`service/core`)	The business logic: orchestration, transactions, rule checks, calling repositories, mapping entities ↔ DTOs, raising exceptions.	Know anything about HTTP
Repository	Spring Data JPA access to one aggregate — derived and custom queries.	Hold business rules
Entity	The JPA domain model (persisted state).	Leave the service boundary
DTO (`dto/request`, `dto/response`)	The API's input and output shapes.	Carry persistence concerns
Mapper (MapStruct)	Convert entities ↔ DTOs at the service boundary.	—
Cross-cutting (`security`, `exception`, `validation`, `config`, `job`, `result`)	Auth, structured errors, input validation, configuration, scheduled jobs, the response envelope.	—

The golden rule: entities never reach the controller, and controllers never hold logic.

The response envelope

Every endpoint returns a generic `Result<T>` in Java — but **the envelope is not what goes over the wire**. `ResultSerializer` unwraps it: on success the body is the bare `data`, on failure the bare `Error`. The HTTP status is the success flag.

`200` — `GET /brands/1`:

```
{
  "id": 1,
  "name": "Monster Energy",
  "introducedInVersion": "1.0.0"
}
```

`404` — `GET /brands/999`:

```
{
  "name": "ElementNotFoundException",
  "message": "BrandEntity not found for id = 999.",
  "status": 404,
  "timestamp": "2026-11-12 20:52:52"
}
```

- This is why the app types its calls directly as `http.get<Brand>(...)` — there is no `.data` to unwrap.
- Collection responses add an **`X-Total-Count`** header; creations add a **`Location`** header.

API surface

Resource paths are centralised in `ApiPaths` and exposed under the API base (for example `/api/drinks`). Around **45 endpoints** across ~14 resources (28 `GET`, 9 `POST`, 4 `DELETE`, 3 `PUT`, 1 `PATCH`):

Resource	Path
Drinks	`/drinks` (incl. bulk `import`, `import/diff`, `import/sync`)
Cans • Can formats	`/cans` • `/can-formats`
Brands	`/brands`
Flavors • Flavor categories	`/flavors` • `/flavor-categories`
Colors • Drink categories	`/colors` • `/drink-categories`
World Dex	`/world-dex`
Users • Unlocked drinks	`/users` • `/user-unlocked-drinks`
App versions • Reports	`/app-versions` • `/reports`
Admin SQL console	`/admin/...`

Interactive docs live at **Swagger UI** (`/swagger-ui.html`). The exact request/response contracts belong to the **API / Contracts** handbook.

Authorization

Endpoints are guarded with `@PreAuthorize` and `SecurityExpressions`:

Rule	Used for
`USER_OR_ADMIN`	Authenticated reads (the catalogue, the World Dex)
`ADMIN_ONLY`	Catalogue writes and admin operations
`PERMIT_ALL`	The few public endpoints
`@securityAccess.canAccessUserId(...)`	User-owned data — a user may only touch their own records

Identity itself comes from Firebase (see Architecture & Overview).

Services & business logic

The domain logic sits in `service/core` — one service per resource (`DrinkService`, `BrandService`, `WorldDexService`, `UserService`, …) — plus supporting services: `ImageService` (analysis), `UploaderService` (media), `ReportService`, and `SqlConsoleService`.

Error handling

Every exception extends a common `CustomException`, which **carries its own `HttpStatus`** and serialises into the `Result` envelope's `error` field (a `dto/response/Error` holding the message and status code). Exceptions fall into two families.

API errors (`exception/api`) — request and data problems:

Exception	HTTP	Raised when
`ElementNotFoundException`	404	A referenced entity does not exist
`InvalidFieldValueException`	400	A field has an invalid format or value
`MissingRequiredFieldException`	400	A required field is absent
`UnsupportedFieldException`	400	An unknown/unsupported field is used
`InvalidImportPayloadException`	400	A bulk-import payload is malformed
`SqlExecutionException`	400	A SQL-console query fails
`ImmutableFieldModificationException`	409	A create-only field is changed after creation
`UniqueConstraintViolationException`	409	A uniqueness constraint is violated
`MissingConfigurationException`	422	Required configuration (e.g. an app version) is missing

Business errors (`exception/business`) — domain-rule violations:

Exception	HTTP	Raised when
`MissingRequiredCanException`	400	A drink is saved without any can
`MissingRequiredLocalizationException`	400	A drink is saved without any localization
`InvalidDexPositionException`	400	A World Dex position is out of range
`InvalidEditionConfigurationException`	400	An edition / variant is misconfigured
`DrinkAlreadyHasPredecessorException`	409	A drink is linked as predecessor to two drinks
`ImageAnalysisException`	422	Image analysis fails

In short: `api` exceptions describe what is wrong with the request; `business` exceptions describe which domain rule was broken.

An exception knows how to turn itself into a response, which is what lets a controller reply with a one-liner (`return e.getResponse();`):

```
// CustomException (abstract, extends RuntimeException) — the base of every
api / business exception
public <T> ResponseEntity<Result<T>> getResponse() {
    Error errorDTO = new Error(
            getClass().getSimpleName(),   // "ElementNotFoundException"
            getMessage(),
            status.value()                // the HttpStatus given in the
constructor
    );
    return ResponseEntity.status(status).body(new Result<>(errorDTO));
}
```

A new rule therefore only needs a new subclass that passes its status up:

```
public class UniqueConstraintViolationException extends CustomException {

    public UniqueConstraintViolationException(Class<?> entity, String field, String
value) {
        super(
                "A %s with %s = '%s' already exists."
                        .formatted(entity.getSimpleName(), field, value),
                HttpStatus.CONFLICT
        );
    }
}
```

Validation

Request DTOs are validated with Bean Validation, wired through `ValidationConfig`. Invalid input surfaces as an `api` error rather than reaching the service layer.

Scheduled jobs

Two background jobs run on a schedule:

- `UnverifiedUserCleanupJob` — removes users who never verified their account.
- `WorldDexHistoryPruneJob` — trims old World Dex history.

Configuration

Configuration lives in `config` and is profile-aware (`dev` / `prod`): `OpenApiConfig` (Swagger), `CorsConfig` (allowed origins), `JacksonConfig` (JSON), `SqlConsoleConfig`, and `ValidationConfig`. To run it locally, see Local Installation; environment specifics belong to the **Deployment** handbook.

Conventions

- Controllers return `ResponseEntity<Result<T>>`; paths come from `ApiPaths` constants.

- Entities never leave the service boundary — DTOs (`dto/request`, `dto/response`) cross it via MapStruct mappers.
- Every endpoint declares its access with `@PreAuthorize`.
- Errors are structured exceptions (`api` / `business`) surfaced through the `Result` envelope.

Where to go next

- The data it persists → Database
- The endpoints it exposes → API / Contracts
- How it's secured → Security
- How it's tested → Testing



CANDEX HANDBOOK

Front-end

The Candex front-ends — the mobile app (Angular + Ionic + NgRx) and the admin dashboard (Angular + Material) — their layers, state, and conventions.

Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

Front-end

Candex has two Angular + Ionic front-ends that consume the same Back-end: the **mobile app** users hold, and the **admin dashboard** that keeps the catalogue accurate. For the system around them, see Architecture & Overview.

Mobile app (`candex-app`)

Angular 20 + Ionic 8 + Capacitor 7, built with **standalone components** (no NgModules), feature-structured, with **NgRx** for state.

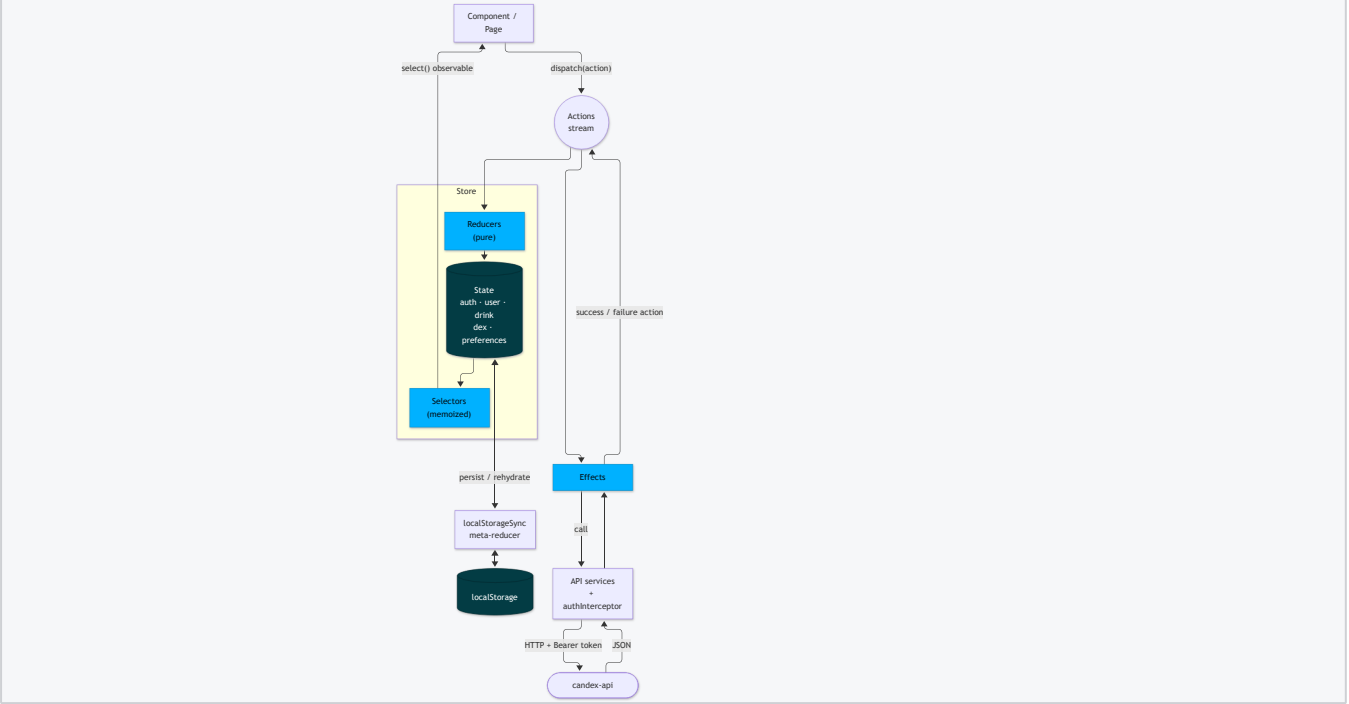
Layers & their roles

Layer	Folder	Role
Pages	`pages`	Routed screens: the auth flow, the `tabs` shell (home / dex / profile), drink details, and settings
Components	`components`	Reusable UI pieces shared across pages
Services	`services`	Side-effects & integrations: the API client (`api`), plus `auth`, `storage`, `theme`, `language`, `upload-care`, `haptics`, `native-app`, `dex-navigation` / `dex-ui`, `ui-alert`, and `app-initializer`
Store	`store`	NgRx state — one slice per domain (see below)
Guards	`guards`	Route protection (`auth`, `app`) — keep unauthenticated users out of protected screens
Interceptors	`interceptors`	`auth.interceptor` attaches the Firebase token to every API request
Pipes	`pipes`	View formatting — e.g. `dynamic-locale-date` for localized dates

The principle mirrors the back-end: **components render and dispatch; services and the store do the work**. A component never calls the API directly.

State with NgRx

State is split into slices — `app`, `auth`, `dex`, `drink`, `preferences`, `user` — each a complete NgRx unit (`actions`, `reducer`, `effects`, `selectors`, `state`). The flow is one-way:



The rules behind the picture:

- **Components only dispatch and select.** Every action goes into a single stream; **reducers** and **effects** both listen to it. A component never knows which of the two handles an action.
- **Reducers are pure** — `(state, action) → new state`, no I/O. Collections are `@ngrx/entity` adapters (`drinksAdapter` for the catalogue, `ownedDrinksAdapter` for the user's collection).
- **Effects own the side-effects** — HTTP, Firebase, navigation, toasts, haptics — and always answer with a new action (`…Success` / `…Failure`), never by writing to state themselves.
- **Slices react to each other through actions**, not imports: `drink` effects emit `tasteDrinkSuccess`, and it is the `user` reducer that records the owned drink.
- **Selectors join slices**: `DrinkSelectors.selectDrinks` merges the catalogue (`drink`) with the owned drinks (`user`) into `DrinkWithUnlock`, so each drink carries its `unlockedAt` without being stored twice.
- **Persistence** — the `localStorageSync` meta-reducer (in `main.ts`) mirrors the `auth`, `user`, `drink`, `dex` and `preferences` slices to `localStorage` and rehydrates them at boot, so a relaunch shows the Dex instantly and `AppActions.loadApp` refreshes it in the background. Ionic Storage (`StorageService`) is used separately, for non-store values.

An action, end to end

Tapping “tasted” on a drink:



There is no optimistic update: the drink only shows as tasted once the API has confirmed it.

The same action in code

Each step of the diagram is a few lines in its own file. The **action** names the intent and carries only what is needed (`store/drink/drink.actions.ts`):

```
const tasteDrink = createAction(
  '[Drinks] Taste Drink',
  props<{ drinkId: number }>()
);
```

The **page** reads the store through a selector and dispatches — it never calls the API:

```
private readonly drinks = toSignal(
  this.store.select(DrinkSelectors.selectDrinks),
  { initialValue: [] }
);

private dispatchTaste(drinkId: number): void {
  this.store.dispatch(DrinkActions.tasteDrink({ drinkId }));
}
```

The **effect** does the I/O and answers with a new action. `concatLatestFrom` reads the user id from the store only when the action arrives; `mergeMap` lets two drinks be tasted in parallel:

```
tasteDrink$ = createEffect(() => {
  return this.actions$.pipe(
    ofType(DrinkActions.tasteDrink),
    concatLatestFrom(() => this.store.select(UserSelectors.selectUserId)),
    filter(([, userId]) => Boolean(userId)),
    mergeMap(([{ drinkId }, userId]) =>
      this.userUnlockedDrinkService.addUnlockedDrink(userId as number,
drinkId).pipe(
        map((unlocked) => DrinkActions.tasteDrinkSuccess({
          drinkId,
          unlockedAt: unlocked.unlockedAt,
        })),
        catchError((error) => of(DrinkActions.tasteDrinkFailure({ drinkId, error
})))
      )
    )
  );
});
```

The **reducer** of another slice (`user`) records the result — pure, no I/O:

```
on(DrinkActions.tasteDrinkSuccess, (state, { drinkId, unlockedAt }): UserState
=> ({
  ...state,
  ownedDrinks: ownedDrinksAdapter.upsertOne({ drinkId, unlockedAt },
state.ownedDrinks),
})),
```

And the **selector** joins both slices, so every subscriber of `selectDrinks` sees the new `unlockedAt`:

```
const selectDrinks = createSelector(
  selectRawDrinks,
  UserSelectors.selectOwnedDrinkEntities,
  (drinks, owned): DrinkWithUnlock[] =>
    drinks.map((drink) => ({
      ...drink,
      unlockedAt: owned[drink.id]?.unlockedAt ?? undefined,
    }))
);
```

Attaching the token

Services never add the `Authorization` header themselves: `auth.interceptor.ts` does it for every request aimed at the API, and lets everything else through untouched:

```
export const authInterceptor: HttpInterceptorFn = (request, next) => {
  if (!request.url.startsWith(environment.apiUrl)) {
    return next(request);
  }

  return from(whenAuthReady(firebase.auth())).pipe(
    switchMap((user) => {
      if (!user) {
        return next(request); // guest: public endpoints only
      }

      return from(user.getIdToken()).pipe(
        switchMap((token) => next(request.clone({
          setHeaders: { Authorization: `Bearer ${token}` },
        })))
      );
    })
  );
};
```

`getIdToken()` returns the cached token and refreshes it on its own when it is about to expire, so the interceptor never has to manage expiry.

Navigation

Routes are standalone: the **auth** flow (`welcome`, `register`, `login`, `complete-profile`, `verify-email`, `two-factor`), the **tabs** shell (`home`, `dex`, `profile`), `drink-details/:drinkId`, and **settings** (`account`, `preferences`, `about`).

Cross-cutting concerns

- **Localization** — `ngx-translate` with a `language` service, and a `dynamic-locale-date` pipe for dates.
- **Theming** — a `theme` service drives light / dark.
- **Native** — Capacitor plugins (haptics, keyboard), a `native-app` service, and Firebase Authentication through Capacitor.
- **Media** — image upload via Uploadcare (`upload-care` service) and cropping via `ngx-image-cropper`.

Admin dashboard (`candex-dashboard`)

Angular 20 + Ionic 8 + **Angular Material**, used to manage the catalogue. It is deliberately simpler than the app: **service-based state** (no NgRx), organised into `components`, `pages`, `services`, `interceptors`, and `interfaces`. An interceptor attaches the admin credentials to API calls, and it talks to the same REST API. Its SQL console is backed by the API's read-only role (see Database & Model).

Where to go next

- Look & feel, components, tokens → **UI/UX (+ Design System)** *(planned)*
- The API the front-ends consume → Back-end
- The big picture → Architecture & Overview



CANDEX HANDBOOK

API / Contracts

The Candex REST API reference — conventions, the response envelope, status codes, headers, authentication, and the endpoints per resource.

Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

API / Contracts

This is the contract reference for `candex-api`. It builds on Back-end (how the API is built) and lists the endpoints per resource. The **live, exhaustive contract** — request/response schemas, examples, and try-it-out — is **Swagger UI** at `/swagger-ui.html`; this page is the stable map around it.

Conventions

- **Envelope** — every response is a `Result<T>` in Java, serialized as the bare payload on success or the bare `Error` on failure (see Back-end).
- **Status codes** — come from typed exceptions (`404` / `400` / `409` / `422`…); see Back-end error handling.
- **Headers** — list responses carry `X-Total-Count`; creations return `201` with a `Location` header.
- **Authentication** — send a Firebase ID token as `Authorization: Bearer <token>`.
- **Access** — per endpoint, in the tables below: `USER_OR_ADMIN` = any authenticated user, `ADMIN` = admin only, `PUBLIC` = no auth, **own** = the authenticated user may only act on their own records.
- **Base path** — resources are served under the API base (for example `/api/drinks`).

Calling the API

A read sends the Firebase ID token and gets the bare DTO back:

```
curl https://<api-host>/api/users/firebase/4q8ESFox1Scd8WMT438M5UlZGMy1 \
  -H "Authorization: Bearer $FIREBASE_ID_TOKEN"
```

```
{
  "id": 42,
  "firebaseUid": "4q8ESFox1Scd8WMT438M5UlZGMy1",
  "username": "therealinvestigator",
  "displayedName": "Investigator",
  "email": "investigator@candex.app",
  "countryCode": "FR",
  "emailVerified": true,
  "joinedInVersion": "1.1.10"
}
```

Registration (`POST /users`) is what the app sends right after creating the Firebase account. `id`, `emailVerified` and `joinedInVersion` are set by the API, never by the caller:

```
{
  "firebaseUid": "4q8ESFox1Scd8WMT438M5UlZGMy1",
  "username": "therealinvestigator",
  "displayedName": "Investigator",
  "email": "investigator@candex.app",
  "countryCode": "FR",
  "birthDate": "2001-07-15"
}
```

It answers `201 Created`, a `Location: /users/42` header, and the created user. Unlocking a drink is the smallest write in the API:

```
{
  "userId": 42,
  "drinkId": 7
}
```

It answers with the stored unlock and its server-side timestamp:

```
{
  "userId": 42,
  "drinkId": 7,
  "unlockedAt": "2026-11-12T20:52:52"
}
```

Every failure has the same shape, whatever the endpoint — here, registering a username that is taken:

```
{
  "name": "UniqueConstraintViolationException",
  "message": "A UserEntity with username = 'therealinvestigator' already exists.",
  "status": 409,
  "timestamp": "2026-11-12 20:52:52"
}
```

The client can branch on `status` for the category of error and on `name` for the exact case.

Catalogue

Drinks — `/drinks`

Method	Path	Access	Purpose
GET	/drinks	USER_OR_ADMIN	List drinks
GET	/drinks/{id}	USER_OR_ADMIN	A single drink
POST	/drinks	ADMIN	Create a drink
POST	/drinks/import	ADMIN	Bulk import drinks
POST	/drinks/import/diff	ADMIN	Preview an import against the current catalogue
POST	/drinks/import/sync	ADMIN	Apply an import
PUT	/drinks/{id}/shift	ADMIN	Move the drink's World Dex position
PUT	/drinks/{id}/swap	ADMIN	Swap two World Dex positions
DELETE	/drinks/{id}	ADMIN	Delete a drink

Cans — /cans

Method	Path	Access
GET	/cans • /cans/{id}	USER_OR_ADMIN
POST	/cans	ADMIN
DELETE	/cans/{id}	ADMIN

Reference data (reads)

Catalogue reference resources expose list and by-id reads: **Brands** /brands, **Flavors** /flavors and **Flavor categories** /flavor-categories, **Colors** /colors (+ /colors/extract), **Drink categories** /drink-categories, **Can formats** /can-formats, and **App versions** /app-versions (+ /app-versions/current).

World Dex — `/world-dex`

Method	Path	Access	Purpose
GET	`/world-dex/drinks/{id}/history`	USER_OR_ADMIN	A drink's position history
GET	`/world-dex/drinks/{id}/stats`	USER_OR_ADMIN	A drink's Dex stats
GET	`/world-dex/snapshot`	USER_OR_ADMIN	Current Dex snapshot
GET	`/world-dex/movers`	USER_OR_ADMIN	Recent movers
GET	`/world-dex/positions/{position}/occupants`	USER_OR_ADMIN	Who sits at a position

Users & collection

Users — `/users`

Method	Path	Access	Purpose
GET	/users	ADMIN	List users
GET	/users/{id}	own	A user
GET	/users/firebase/{firebaseUid}	own	A user by Firebase UID
GET	/users/{id}/drinks	own	A user's unlocked drinks
GET	/users/exists	PUBLIC	Existence check
POST	/users	PUBLIC	Register
PUT • PATCH	/users/{id}	own	Update a user
DELETE	/users/{id}	own	Delete a user

Unlocked drinks — `/user-unlocked-drinks`

Method	Path	Access	Purpose
GET	/user-unlocked-drinks/{userId}	own	A user's unlocks
POST	/user-unlocked-drinks	own	Unlock a drink
DELETE	/user-unlocked-drinks/{userId}/{drinkId}	own	Remove an unlock

Admin & operations

- **SQL console** — `POST /admin/...` (ADMIN): run a read-only query through the console role (see Database & Model).
- **Reports** — `POST /reports`: generate a report.

The bulk-import flow

Importing a batch of drinks is a three-step, admin-only pipeline:

1. `POST /drinks/import` — submit the payload.
2. `POST /drinks/import/diff` — preview the changes against the current catalogue.
3. `POST /drinks/import/sync` — apply them.

Where to go next

- How the API is built → Back-end
- Live schemas & try-it-out → **Swagger UI** (`/swagger-ui.html`)



CANDEX HANDBOOK

UI / UX

The look, feel, and interaction patterns of the Candex mobile app — design tokens, theming, the component library, navigation, and UX conventions.

Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

UI / UX

Candex's interface is an **Ionic 8 + Angular 20** mobile app. This handbook is the design and interaction side; for how the front-end code is structured, see Front-end.

Design tokens

Colours, typography, and surfaces are defined once as **Ionic CSS variables** and reused everywhere, so the brand stays consistent across every screen.

primary
#00B1FF
Candex blue — primary actions, accents

secondary
#ffffff
White

tertiary
#033C44
Deep teal — secondary emphasis

success
#2dd36f
Success state

warning
#ffc409
Warning state

danger
#eb445a
Danger / error state

Each colour ships with its `-shade`, `-tint`, and `-contrast` variants, so text always meets its background. Typography is **Rubik**, self-hosted from `assets/fonts` (Regular, Medium, Bold, and the rest of the family) and wired as `--ion-font-family`, so the app renders identically offline and across platforms.

Typography

The app's typeface is **Rubik**, self-hosted from `assets/fonts` and set as `--ion-font-family`. Three weights are loaded and carry the whole hierarchy: **Regular (400)** for body text, **Medium (500)** for emphasis and labels, and **Bold (700)** for headings and strong text.

Text sizing follows **Ionic's built-in typography** (the app imports `typography.css`) rather than a bespoke scale: a **16px (1rem)** base, with headings stepping down from `<h1>` to `<h6>` on Ionic's default scale. In short, the app owns the *family*; Ionic owns the *sizes* — which keeps text consistent and respects the platform's accessibility (dynamic type) settings.

Styling conventions

Styles are written in SCSS and follow a few strict rules, so the codebase stays uniform:

- **Cascade, always** — styles are written as nested SCSS that leans on the cascade, not flat, repeated selectors.
- **A fixed spacing scale** — margins, paddings, gaps, and offsets use only **2, 4, 8, 16, 32** px. `12` and `24` are allowed, but only in rare, specific cases.
- **Global variables for shared values** — colours, fonts, and text sizes always come from global variables (the design tokens), never hard-coded literals.
- **No floating CSS** — no inline styles in templates and nothing styled outside a `.scss` file; every rule lives in SCSS.
- **Consistency above all** — the same thing is done the same way everywhere.

Here is what that looks like, trimmed from `update-required.component.scss`: one root block per component, children nested with `&__`, colours only through theme variables (so light and dark work for free), spacing on the scale:

```
.update-required {
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 32px 24px;
  background: var(--ion-background-color);

  &__icon {
    margin-bottom: 16px;
    color: var(--ion-color-primary);
  }

  &__text {
    margin: 0 0 24px;
    color: var(--ion-color-medium);
  }
}
```

Theming — Light, Dark, System

The app offers three themes through a `ThemeService`: **Light**, **Dark**, and **System** (which follows the device setting). The choice is applied by setting a `theme` attribute on `<body>`, and the token blocks `body[theme=light]` / `body[theme=dark]` swap the palette underneath — every component reacts automatically because it reads the same variables.

Component library

On top of Ionic's own components (buttons, cards, lists, modals, tabs) sit a set of **shared components** that carry the app's specific patterns:

- **Content** — `drink-card` renders a can consistently across the Dex and lists.
- **Feedback** — `banner` and `error-banner` surface inline messages; `form-errors` renders field-level validation under inputs.
- **Focused tasks (modals)** — `avatar-cropper-modal`, `fullscreen-image-modal`, `report-problem-modal`, and the Google account flows (`google-account-modal`, `google-link-prompt-modal`, `google-photo-choice-modal`) keep one task in view at a time.

- **Auth** — `social-auth-buttons` provides the social sign-in entry points.

Icons come from **Ionicons**, the set bundled with Ionic.

Navigation

The app runs on a **three-tab shell** (`tabs.page`):

- **Dex** — the collection, and the app's default landing tab.
- **Search** — a tap action that opens search rather than a static tab page.
- **Profile** — the user's account and settings.

Pages are **lazy-loaded standalone components** (`loadComponent`), so each screen loads on demand. Outside the tabs, an **auth stack** carries the sign-in journey — `welcome`, `register`, `login`, `verify-email`, `two-factor`, and `complete-profile` — and dedicated routes handle detail and settings views (`drink-details/:drinkId`, `settings`). Authentication itself is covered in Security.

Interaction & content conventions

A few patterns repeat throughout:

- **Modals for focused work** — cropping an avatar, viewing an image fullscreen, linking a Google account, or reporting a problem each happen in a modal, keeping the underlying screen intact.
- **Inline, field-level validation** — forms show errors next to the offending field via `form-errors`, not as disruptive alerts.
- **Banners for status** — transient and error messages appear as banners rather than blocking dialogs.

Internationalization

The UI is fully translated with **ngx-translate**: strings are externalized to `en.json` and `fr.json` and rendered through the `| translate` pipe (`tabs.dex`, `tabs.search`, …), so English and French stay in step and new languages are a matter of adding a locale file.

Where to go next

- The code behind these screens → Front-end
- What the app is for → Understanding Candex
- Sign-in and permissions → Security



CANDEX HANDBOOK

Security

How the API authenticates callers and authorizes requests — Firebase for app users, an interim Basic Auth for the admin, and method-level authorization rules.

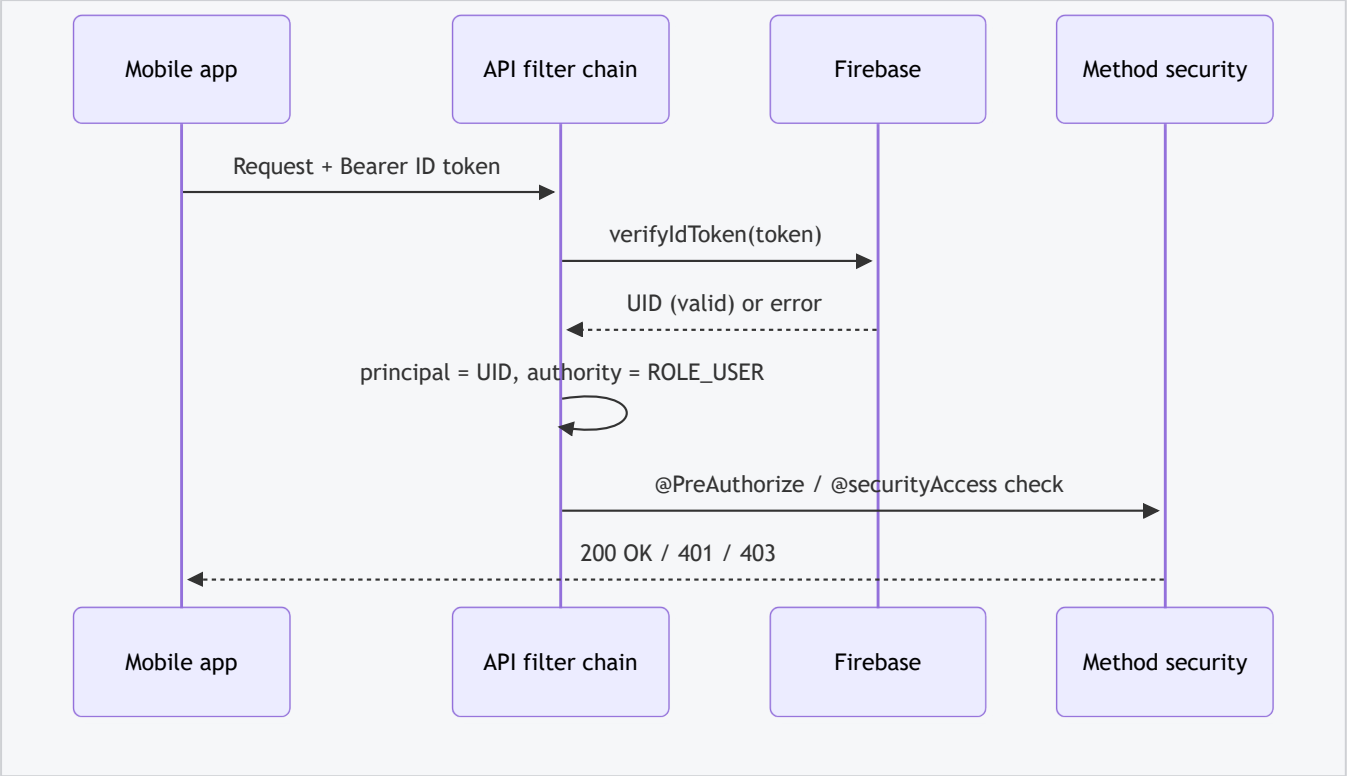
Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

Security

Security is enforced in the **API**. Authentication decides *who* is calling; authorization decides *what* they may do. The HTTP layer stays deliberately thin — it sets the authentication mechanism and a few public routes — while the real business rules live in method security, next to the endpoints they protect.

Request authentication at a glance



Authentication

Two mechanisms coexist.

Firebase ID tokens are the primary path, for app users. A request carries `Authorization: Bearer <token>`; the `FirebaseAuthenticationFilter` verifies it (`FirebaseAuth.verifyIdToken`) and, on success, sets a Spring Security authentication whose **principal is the Firebase UID** and whose authority is `ROLE_USER`. Verification is non-blocking: if the token is missing, invalid, expired, or revoked, the filter simply clears the

context and lets authorization reject the request downstream. Firebase is the source of truth for app identity.

The whole filter fits in a few lines — note that it never rejects a request itself:

```
// FirebaseAuthenticationFilter (a OncePerRequestFilter)
String authorization = request.getHeader("Authorization");

if (authorization != null && authorization.startsWith("Bearer ")) {
    try {
        Authentication authentication =

firebaseAuthenticationService.authenticate(authorization.substring(7));

        if (authentication != null) {
            SecurityContextHolder.getContext().setAuthentication(authentication);
        }
    } catch (FirebaseAuthException exception) {
        SecurityContextHolder.clearContext(); // invalid token = anonymous, not an
error here
    }
}

filterChain.doFilter(request, response); // @PreAuthorize decides later
```

Interim HTTP Basic Auth covers the administrator. An in-memory user store (`InMemoryUserDetailsManager`) holds two accounts loaded from environment variables — a standard `user` (`ROLE_USER`) and an `admin` (`ROLE_ADMIN`) — with BCrypt-hashed passwords. The code documents this as a temporary bridge, intended to be replaced by JWT-based authentication later.

The filter chain is **stateless** (`SessionCreationPolicy.STATELESS`), **CSRF is disabled** (there are no server sessions or cookies to protect), and **CORS** is handled by a dedicated `CorsFilter` with a fixed allow-list of origins (`CorsOrigins`).

Roles

There are two authorities — `ROLE_USER` and `ROLE_ADMIN` (`SecurityRoles`).

Authorization

Authorization runs in two layers.

At the HTTP layer (`SecurityConfig`), only a handful of rules are declared:

- **Public** — `/error`, `/actuator/health`, user registration (`POST /users`), and the existence check (`GET /users/exists`).
- **Admin only** — Swagger UI and the OpenAPI docs (`/swagger-ui/**`, `/v3/api-docs/**`) and the actuator (`/actuator/**`).
- **Everything else** — authenticated.

At the method layer (`@EnableMethodSecurity`), each endpoint carries a `@PreAuthorize` rule, built from reusable `SecurityExpressions`:

- `USER_OR_ADMIN` — any authenticated user or admin.
- `ADMIN_ONLY` — admin authority required (catalogue writes, imports, admin operations).
- `PERMIT_ALL` — explicitly open.

Ownership-scoped endpoints go through the `@securityAccess` bean (`SecurityAccess`) rather than a role alone, e.g. `@PreAuthorize("@securityAccess.canAccessUserId(authentication, #id)")`. The rule is consistent: an **admin bypasses ownership**, otherwise the caller's Firebase UID (or user id) must match the resource's owner. The same bean also guards Firebase-UID access (`canAccessFirebaseUid`) and unlocked-drink management (`canManageUserUnlockedDrink`).

In code, a route states its rule in one annotation, and the ownership check is a plain Java method on the bean:

```
@GetMapping("/{id}")
@PreAuthorize("@securityAccess.canAccessUserId(authentication, #id)")
public ResponseEntity<Result<User>> getUserById(@PathVariable Integer id) { ... }
```

```
// SecurityAccess
public boolean canAccessUserId(Authentication authentication, Integer userId) {
    if (authentication == null || !authentication.isAuthenticated() || userId ==
null) {
        return false;
    }

    if (hasAuthority(authentication)) { // ROLE_ADMIN
        return true;
    }

    // The principal's name is the Firebase UID set by the filter.
    return Objects.equals(
            userService.getFirebaseUidByUserId(userId),
            authentication.getName()
    );
}
```

`#id` refers to the method parameter of the same name, so the check runs with the value from the URL before the controller body executes.

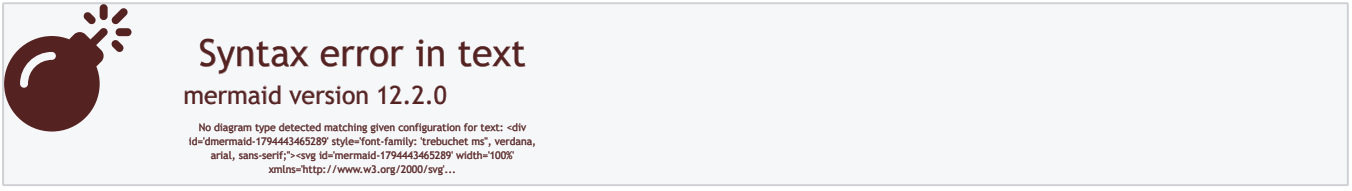
User lifecycle

An app user exists **twice**: as a **Firebase account** (identity, password, providers, email verification) and as a **`users` row in PostgreSQL** (profile, collection, role in the domain). The two are linked by `users.firebase_uid`. Every step of the lifecycle has to keep them in sync — the app orchestrates creation (`AuthEffects`), the API orchestrates deletion (`UserService`, `UnverifiedUserCleanupJob`).

State	Firebase account	`users` row	App `AuthStatus`
Guest	—	—	`guest` / `anonymous`
Registering	being created	being created	`registering`
EmailUnverified	exists, `emailVerified = false`	exists, `email_verified = false`	`emailUnverified`
SocialProfilePending	exists (provider)	—	`socialProfilePending`
Authenticated	exists, verified	exists	`authenticated`
Purged / Deleted	deleted	deleted	back to `anonymous`

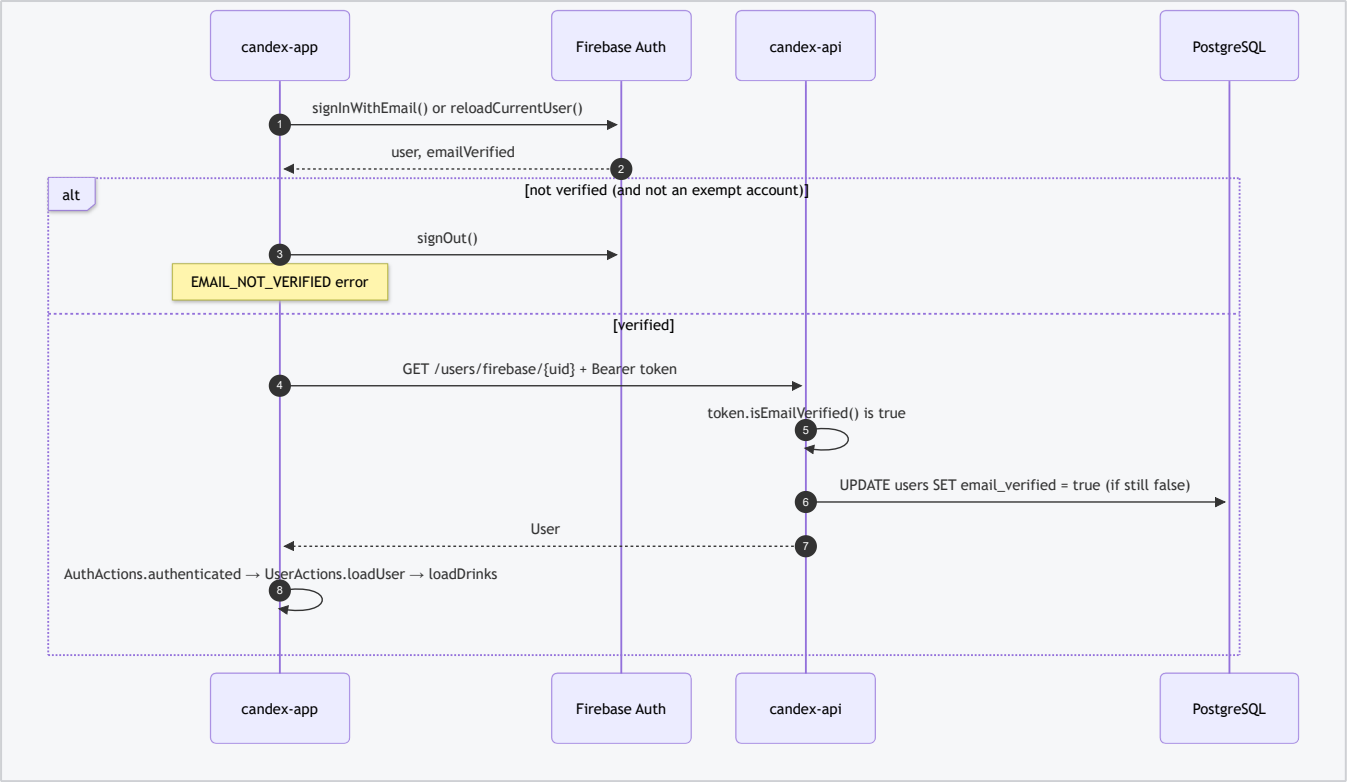
Registration (email & password)

The app creates both sides and **rolls both back** if anything fails, so neither an orphan Firebase account nor an orphan row survives a failed sign-up.



Verification, sign-in and the `email_verified` flag

Firebase owns verification; PostgreSQL keeps a **copy** (shown on the dashboard and read by the cleanup job). The copy is synced lazily: whenever the app calls `GET /users/firebase/{uid}` with a token whose `email_verified` claim is true, `UserService.getUserByFirebaseUid` flips the row to `true`.



Social sign-in (Google / Apple) follows the same `GET /users/firebase/{uid}` call: a `200` signs the user in; a `404` means "first time" — unless `GET /users/exists?field=email` says the email is already registered with another provider, in which case the app signs out with `ACCOUNT_EXISTS_DIFFERENT_PROVIDER` instead of creating a duplicate.

Deletion

Both deletion paths delete the **row first, then the Firebase account**, inside one transaction: if Firebase fails (other than "user not found"), the exception rolls the row back, so the two never drift apart. The Uploadcare avatar is removed last, best-effort.

- **By the user** — `POST /users/me/deletion-code` emails a 6-digit code (hashed in `account_deletion_code`, valid 1 h, 5 attempts, 1 resend per minute); `DELETE /users/me` with that code calls `UserService.deleteUserById`.
- **By the cleanup job** — `UnverifiedUserCleanupJob` runs daily at 03:30 and looks for rows still `email_verified = false` after 72 h (`candex.security.unverified-user-retention`). For each, it asks Firebase first: if the user did verify, the stale flag is re-synced and the account kept; otherwise both sides are deleted. Accounts listed in `candex.security.cleanup-protected-emails` are skipped.
- **By an admin** — `DELETE /users/{id}` (`canDeleteUserId`), same `deleteUserById` path.

Secrets

No credential or key is committed. The Firebase service account is loaded at runtime from a configured file path (`candex.firebase.config-path`), never from the repository. The Basic-auth accounts (`candex.security.*`) and the database credentials come from environment variables, per Spring profile — the repositories ship only `.env.example` files with placeholders. See Local Installation for setting them locally and Deployment for the deployed environments.

On the client side

The mobile app signs users in with **Firebase Authentication** and calls the API with the resulting ID token in the `Authorization` header; the API trusts Firebase to have authenticated the user and derives identity from the verified token. See Architecture for how the pieces connect.

Where to go next

- The endpoints these rules protect → API / Contracts
- The layers behind them → Back-end
- The big picture → Architecture



CANDEX HANDBOOK

Deployment

How Candex ships — the API as a Docker Compose stack per environment, CI/CD, the reverse proxy, and the mobile app's store releases.

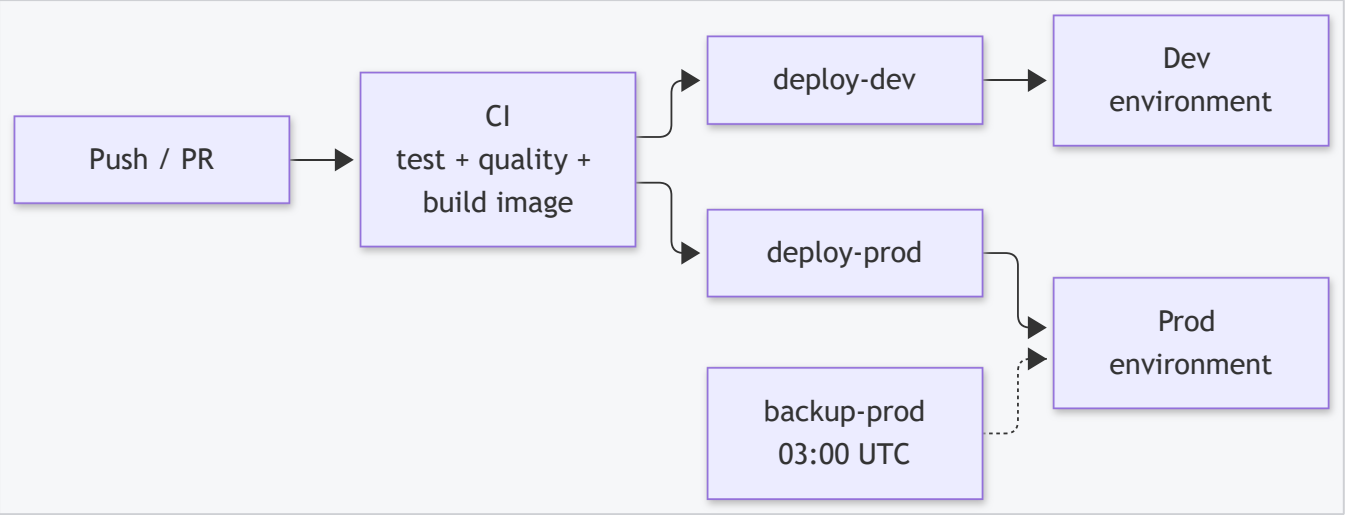
Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

Deployment

This is how Candex is built, shipped, and operated. The API runs as a **Docker Compose** stack per environment; the mobile app ships to the stores with **Fastlane**. To run it on your machine, see Local Installation; for the system around it, Architecture & Overview.

The pipeline at a glance



Environments

Each environment is its own Compose stack (an API container + a PostgreSQL 17 container):

Environment	Compose file	Exposure
local	`compose.local.yaml`	Throwaway stack via `scripts/candex.sh` (Local Installation)
dev	`compose.dev.yaml`	`api-dev` + `db-dev`; API on host `8081`, debugger on `5005`
prod	`compose.prod.yaml`	`api-prod` + `db-prod`; **no published ports** — reached through the reverse proxy

Each API container runs with `SPRING_PROFILES_ACTIVE` set to its environment, restarts `unless-stopped`, and keeps rotated `json-file` logs.

The API image

The API is built by a multi-stage `Dockerfile` — compiled with Maven, then run on a lean JRE:

```
FROM maven:3.9.9-eclipse-temurin-21 AS build
# ... mvn -B -DskipTests package
FROM eclipse-temurin:21-jre
COPY --from=build /app/target/*.jar /app/app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Configuration is entirely environment-driven (database, security, SQL console, Uploadcare) — see Local Installation for the variables. Nothing secret is baked into the image.

Operating a stack

`scripts/candex.sh` operates a running dev or prod stack from the server — without stopping it:

```
./scripts/candex.sh psql <dev|prod>        # a psql session
./scripts/candex.sh logs <dev|prod>        # follow the API logs
./scripts/candex.sh dump <dev|prod>        # pg_dump the database
./scripts/candex.sh exec <dev|prod> "<SQL>"
```

CI/CD

GitHub Actions drive integration and delivery (`.github/workflows`):

Workflow	Role
`ci.yml`	Quality gate on push / PR — build, tests, lint (and Qodana)
`deploy-dev.yml` • `deploy-prod.yml`	Deploy the API to dev / prod
`backup-prod.yml`	Scheduled production database backup

The dashboard has its own `ci.yml` quality gate.

Reverse proxy & TLS

> **To confirm**
>
> The production API publishes no ports; the public entrypoint and TLS are handled by a **reverse proxy at the server level** (a `CADDY_EMAIL` variable points to Caddy). The exact proxy configuration lives outside these repositories and is not documented here yet.

Mobile app releases

The app ships to the **Play Store** and **App Store** with **Fastlane**, triggered manually from GitHub Actions. Version bump and changelog stay local and human-reviewed — CI only deploys an existing tag:

```
npm run release 1.2.0            # bump + annotated tag x.y.z (message = changelog)
git push && git push origin 1.2.0
# GitHub ▸ Actions ▸ "Deploy to Stores" ▸ Run workflow (store: both / play / appstore)
```

`deploy.yml` (a manual `workflow_dispatch`) builds the chosen tag and submits it through the Fastlane lanes `android deploy` (AAB → Play production) and `ios deploy` (IPA → App

Store Connect).

Where to go next

- Database backups → Backups
- Run it on your machine → Local Installation
- The API itself → Back-end



CANDEX HANDBOOK

Backups

How the Candex production database is backed up — the strategy, the scripts, restore testing, and the CI automation.

Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

Backups

Candex backs up its **production PostgreSQL** database. This handbook covers the strategy, the scripts, how to restore, and how it is automated. For where the database lives, see Database & Model; for the environments, Deployment.

> **Note**
>
> The legacy `candex-backups` repository is the old Firestore-era tool and no longer backs up real data. The current backups are PostgreSQL dumps driven from `candex-api/scripts/backup/`.

Strategy

Only **prod** (`candex_prod`) is backed up — dev is a proving ground. The approach is a daily full dump with grandfather-father-son (GFS) retention, plus a permanent snapshot per release:

- **Daily** full dumps — `KEEP_DAILY = 7`
- **Weekly** — the Sunday dump is promoted — `KEEP_WEEKLY = 5`
- **Monthly** — the 1st-of-month dump is promoted — `KEEP_MONTHLY = 12`
- **Per version** — a labelled snapshot (`--label x.y.z`), **kept forever, never pruned**

Full backups — `daily_backup.sh`

The core script runs a `pg_dump -Fc` (custom format: compressed and selectively restorable) of the whole database, through the running Postgres container:

```
./scripts/backup/daily_backup.sh prod                 # daily full dump
./scripts/backup/daily_backup.sh prod --label 1.1.9   # + a kept version snapshot
```

Each run writes to `backups/<env>/full/`, validates the dump (it must be non-empty and pass `pg_restore --list`), then promotes it into the `daily`, `weekly`, `monthly`, and `version`

tiers as applicable:

```
backups/prod/full/
├── daily/     candex_prod_<timestamp>.dump
├── weekly/    (Sundays)
├── monthly/   (1st of month)
└── version/   candex_prod_v1.1.9.dump   (kept forever)
```

The `backups/` folder is git-ignored — dumps never go into git.

Selective exports — `dump_*.sh`

Alongside the full dumps, per-domain **data-only** exports cover the runtime data that is hard to reconstruct — `dump_catalog_data.sh`, `dump_dex_data.sh`, `dump_user_data.sh` (and `dump_all_runtime_data.sh`). Static reference data is intentionally excluded, because Flyway seeds it (see Database & Model).

Restore & testing

A backup is only as good as a tested restore. `restore.sh` restores a dump into a **throwaway** database so you can verify it, then drop it:

```
./scripts/backup/restore.sh <file.dump> candex_restore_test
# verify, then: dropdb candex_restore_test
```

`pg_restore` must be the same version as the server or newer.

Automation (CI)

Backups run on the self-hosted runner that sits on the production server, via GitHub Actions:

Workflow	When	What
`backup-prod.yml`	Daily at **03:00 UTC** (and manual)	`daily_backup.sh prod`
`deploy-prod.yml`	Before every production deploy	`daily_backup.sh prod --label <version>` — a pre-deploy recovery point

No OS cron is needed — the schedule lives in the workflow.

Storage & off-site

Dumps are kept **on the server** for now; getting them off-host (a second disk or `rsync` to another machine) is a planned improvement. A standalone variant under `ops/backup/` (a `systemd` timer plus `rsync` off-site copy and failure alerting) is available for that server-side setup.

Conventions

- **No secrets in git** — the Postgres password lives in `~/.pgpass` (`chmod 600`); local config (`backup.conf`) and the `backups/` dumps are never committed.
- **Full + selective** — full `-Fc` dumps are the restore path; the per-domain exports are convenience snapshots.
- **Test restores** — a dump that has never been restored is not a backup.

Where to go next

- Where the data lives → Database & Model
- Environments & CI → Deployment



CANDEX HANDBOOK

Testing

How Candex is tested — unit and integration tests on the API, unit and e2e tests on the front-ends, and the CI quality gates.

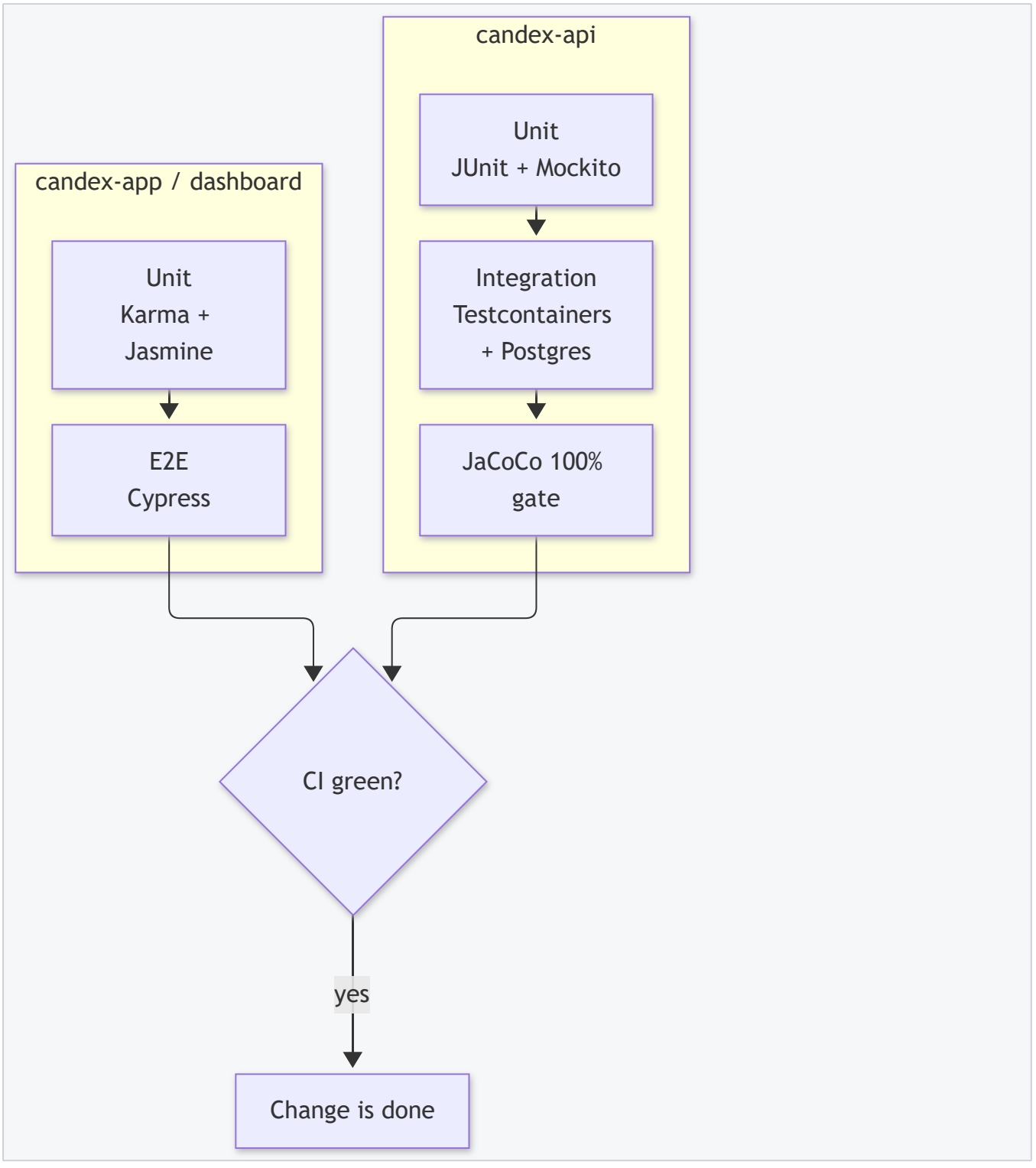
Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

Testing

Testing runs across the whole stack and is enforced in CI. For how the code is organised, see Back-end and Front-end.

The test setup at a glance



Back-end (`candex-api`)

The API has around **90 test classes** on two levels:

- **Unit tests** — **JUnit 5** with **Mockito** for isolated logic.
- **Integration tests** — **Testcontainers** spins up a real **PostgreSQL** container, so tests run against the actual database rather than a mock.

Coverage is a hard gate: **JaCoCo** enforces **100%** at the bundle level (line and branch) — the build fails below the threshold. **Checkstyle** enforces the code style.

```
./mvnw test        # run the tests
./mvnw verify      # tests + JaCoCo coverage gate + Checkstyle
```

Front-ends (`candex-app`, `candex-dashboard`)

Both Angular apps use **Karma + Jasmine** for unit tests, run headless with coverage. Specs live next to the code (`*.spec.ts`) — roughly **80** in the app and **40** in the dashboard. The mobile app also has **Cypress** for end-to-end tests.

```
npm run test       # watch mode, with coverage
npm run test:ci    # headless (ChromeHeadless), single run, with coverage
```

Continuous integration

Every push and pull request runs a **quality gate** (`.github/workflows/ci.yml`) per repository:

Repository	What CI runs
`candex-api`	`mvn clean verify checkstyle:checkstyle` (tests + 100% JaCoCo gate + Checkstyle), then a Docker image build; uploads the Checkstyle and JaCoCo reports
`candex-app` • `candex-dashboard`	`ng test` headless with coverage, plus lint

Qodana adds static code-quality analysis on top.

Conventions

Tests mirror the code they cover, at every level:

- **One unit per file** — a test class (API) or top-level `describe` (front) per class or service, named after it: `@DisplayName("ColorMatcher")`, `describe('SettingsActionsService')`.
- **One method per group** — each method under test gets its own `@Nested` class on the API (`@DisplayName("findNearestAnchor()")`) or a nested `describe('refreshProviderState()')` on the front.
- **One case per test** — a single `@Test` / `it` per scenario, each named as the behaviour it asserts (*"should return semantic match when a ColorGroup zone matches"*, *"dispatches linkGoogle when not yet linked"*). On the API every level carries a `@DisplayName`, so the test report reads as sentences.
- **A smoke test per file** — `contextLoads()` (*"should load correctly"*) on the API, `it('should be created')` on the front, so a wiring or injection failure surfaces first.
- **Shared fixtures, not re-created objects** — reusable test data lives in one place and is imported, never rebuilt in each file: the API's `util/test/fixture` package (`TestApp`, `TestEntity`, `TestGetDTO`, `TestPostDTO` — final classes exposing fixed constants like `FIXED_NOW` and canned entities/DTOs), and the front-ends' `utils/tests/mocks.ts` and test helpers (dashboard: `tests/mocks`).

And across the stack:

- Tests live **beside the code** they cover (`*.spec.ts`; `*Test` on the API).

- Integration tests use **Testcontainers**, not mocks, so they exercise real SQL and migrations.
- The API's **100% coverage** gate means new code ships with its tests — and a change is not done until CI is green.

What a test looks like

An API unit test, from `BrandServiceTest`: the service is built with mocked collaborators, the fixtures (`B_ENTITY_LIST`, `B1`…) come from `util/test/fixture`, and `verifyNoMoreInteractions` proves the service did nothing it should not have:

```
@ExtendWith(MockitoExtension.class)
@DisplayName("BrandService")
class BrandServiceTest {

    @Mock private BrandRepository brandRepository;
    @Mock private BrandMapper brandMapper;

    @InjectMocks
    private BrandService brandService;

    @Nested
    @DisplayName("getAllBrands()")
    class GetAllBrands {

        @Test
        @DisplayName("should return all brands as DTOs")
        void shouldReturnAllBrands() {
            // Arrange

when(brandRepository.getAllBrandsOrderedById()).thenReturn(B_ENTITY_LIST);
            when(brandMapper.toDTO(B_ENTITY_LIST)).thenReturn(B_LIST);

            // Act & Assert
            assertThat(brandService.getAllBrands())
                    .hasSize(3)
                    .containsExactly(B1, B2, B3);

            verify(brandRepository).getAllBrandsOrderedById();
            verify(brandMapper).toDTO(B_ENTITY_LIST);
            verifyNoMoreInteractions(brandRepository, brandMapper);
        }
    }
}
```

A front-end reducer test, from `user.reducer.spec.ts`. Reducers are pure functions, so no `TestBed` is needed — call it with a state and an action, check the state that comes out:

```
describe('tasteDrinkSuccess()', () => {
  it('should add the owned drink to the collection', () => {
    // Arrange
    const action = DrinkActions.tasteDrinkSuccess({
      drinkId: 2,
      unlockedAt: '2026-04-18T20:52:52.110917Z',
    });

    // Act
    const state = userReducer(initialUserState, action);

    // Assert
    expect(state.ownedDrinks.entities[2]).toEqual({
      drinkId: 2,
      unlockedAt: '2026-04-18T20:52:52.110917Z',
    });
    expect(state.error).toBeNull();
  });
});
```

Where to go next

- Code style, conventions, static analysis → Software Quality
- The API → Back-end • The front-ends → Front-end



CANDEX HANDBOOK

Software Quality

How code quality is kept consistent — Checkstyle on the API, ESLint and Prettier on the front-ends, Qodana static analysis, and the conventions behind them.

Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

Software Quality

Quality here means code that stays consistent, simple, and low-complexity — enforced by tooling in CI, not by memory. Tests and coverage are covered separately in Testing; this handbook is about **style, conventions, and static analysis**.

Back-end — Checkstyle

The API is checked against a custom **Checkstyle** config (`checkstyle.xml`) covering naming (`TypeName`, `MethodName`, `MemberName`, `ConstantName`), imports (no star imports, no unused/redundant imports), whitespace, braces (`NeedBraces`), modifier order, and indentation. Lines are capped at **120** characters.

It also caps complexity, so methods stay small and readable:

Rule	Limit
Method length	60 lines
Parameters per method	7
Cyclomatic complexity	10
NPath complexity	200
Anonymous inner class length	20 lines

A companion `checkstyle-suppressions.xml` relaxes a few rules where they add noise: controllers skip Javadoc-on-methods (Spring annotations are self-explanatory), entities may use magic numbers, and generated MapStruct mappers, controllers, and test sources are exempt from line-length and indentation checks.

Checkstyle runs in CI (`mvn … checkstyle:checkstyle`) and the HTML report is uploaded as a build artifact.

Front-ends — ESLint & Prettier

Both Angular apps lint with **ESLint 9** using `@angular-eslint` and `@typescript-eslint`, and format with **Prettier**:

```
npm run lint      # ng lint (ESLint)
npm run format    # prettier --write .
```

`npm run lint` runs in CI for both the app and the dashboard. An `.editorconfig` fixes the shared basics — UTF-8, **2-space** indentation, a final newline, trimmed trailing whitespace, and **single quotes** in TypeScript. The dashboard adds a few extra ESLint plugins (`import`, `jsdoc`, `prefer-arrow`). The mobile app has **husky** available for Git hooks, though linting currently runs manually and in CI rather than on commit.

Static analysis

Two layers of static analysis run on top of linting.

DeepSource runs continuously across all three repositories (`.deepsource.toml`):

- **API** — the Java analyzer (runtime 21), with `test_patterns` narrowed to `**/*Test.java` so the shared fixtures (`TestApp`, `TestEntity`, `TestGetDTO`, `TestPostDTO`, `TestHelper`) aren't flagged as empty test files.
- **Mobile app** — the JavaScript analyzer with the Angular plugin (Node, Jasmine, and Cypress environments); a couple of rules are skipped where Angular's patterns trip them (empty module classes `JS-0327`, required-input aliasing `JS-0585`).
- **Dashboard** — the JavaScript analyzer with the Angular plugin (Node, Jasmine).

JetBrains Qodana adds a second pass (the `qodana.starter` profile), but only on the **API** (`qodana_code_quality.yml`, JVM community linter). On the mobile app a Qodana workflow exists but is **disabled** (`.yml.disabled`), and the dashboard has none. Qodana's quality-gate thresholds in `qodana.yaml` are left commented out, so it reports findings without failing the build on a count *(to confirm)*.

Where quality is enforced

Repository	Style	Analysis
`candex-api`	Checkstyle	DeepSource + Qodana
`candex-app`	ESLint + Prettier	DeepSource
`candex-dashboard`	ESLint + Prettier	DeepSource

The API also has the JaCoCo 100% coverage gate — see Testing. Linting and Checkstyle run in CI on each push and pull request, DeepSource analyses every change continuously, and a change is not done until everything is green. For the full CI picture and the test/coverage gates, see Testing.

Where to go next

- Tests and coverage → Testing
- The code these rules apply to → Back-end • Front-end



CANDEX HANDBOOK

Releasing

How the three Candex projects are versioned and shipped — the mobile app to the stores, the API via its deploy pipeline, and the admin dashboard.

Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

Releasing

Candex is three deployable projects, each versioned and released on its own: the mobile app (`candex-app`), the API (`candex-api`), and the admin dashboard (`candex-dashboard`). For the runtime and infrastructure side, see Deployment.

Mobile app (`candex-app`)

The app carries one version across web, Android, and iOS. A release is cut with the release script:

```
npm run release 1.2.0
```

`scripts/release.js` bumps the version everywhere it lives — `package.json`, the Android `build.gradle` (`versionName` and `versionCode`), and the iOS `project.pbxproj` — updates `CHANGELOG.md`, commits the change, and creates an **annotated git tag** whose message is that version's changelog section. If any step fails it rolls back (restores the files, deletes the tag). It then prints the push command:

```
git push && git push origin 1.2.0
```

Pushing the tag triggers **`github-release.yml`** ("Create GitHub Release"), which publishes the GitHub Release. Shipping to the stores is a separate, manual step: the **"Deploy to Stores"** workflow (`deploy.yml`, `workflow_dispatch`) takes an already-released tag and, via **Fastlane**, builds and submits **Android → Google Play** (an AAB) and **iOS → App Store** (an IPA to App Store Connect).

(Current app version: 1.1.4.)

API (`candex-api`)

The API is a Maven project; its version lives in `pom.xml` (currently 1.1.10). It is released by deploying its Docker image: the **Deployment Development** (`deploy-dev.yml`) and **Deployment Production** (`deploy-prod.yml`) workflows run automatically after CI completes successfully (`workflow_run`), or manually (`workflow_dispatch`). The image build and the Compose stacks are covered in Deployment.

Admin dashboard (`candex-dashboard`)

The dashboard follows the same release-script style: `npm run release` bumps `package.json` and `CHANGELOG.md` and tags the version (web only, no mobile files). Its version is currently 1.0.0. CI runs on every change; there is no store or deploy workflow in the repository, so its deployment is handled elsewhere *(to confirm)*.

Versioning

Each project has its own semantic version and its own `CHANGELOG.md` — the app, the API, and the dashboard move independently.

Where to go next

- The runtime, images, and infrastructure → Deployment
- The checks that gate a release → Testing • Software Quality



CANDEX HANDBOOK

Glossary

The domain vocabulary behind Candex — the terms the app, the data model, and the other handbooks all rely on.

Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

Glossary

Candex has its own vocabulary. These are the terms the app, the Database, and the Back-end all share, grounded in the data model.

Collection & progression

Dex — a user's **personal collection**: the set of drinks they have unlocked. It is the app's home tab and the heart of the experience.

Unlock — the act of adding a drink to one's Dex. A drink can be unlocked only once per user (`UserUnlockedDrink`, keyed on user + drink), and the unlock is timestamped.

World Dex — the **global, ranked list** of drinks shared by everyone: each drink holds a numbered position within the world-wide dex. Distinct from a user's personal Dex.

World Dex history — the record of how World Dex positions changed over time. A database trigger logs each *departure* (the position a drink left) and an *arrival* row marks when a drink first entered, so a drink's position at any past instant can be reconstructed. See Database for the mechanism.

Catalogue

Drink — the central catalogue item: a specific beverage product, with its names, brand and category, flavors, edition flags, lifecycle dates, and the app version it was introduced in. Nutrition and claims are stored separately, per country.

Can — a physical can of a drink, described by its format, colour, country, and language. One drink can have several cans.

Brand — the beverage brand a drink belongs to (e.g. the maker).

Series — a sub-line within a brand — a catalogue sub-family, e.g. Monster's *Ultra* range.

Category (*drink category*) — the kind of beverage (the classification a drink falls under).

Flavor — a flavour tag attached to a drink; a drink can carry several. Flavours are grouped into flavour categories and have localized names (and optional descriptions) per language.

Edition — edition flags on a drink, distinguishing special or limited editions from the standard product.

Predecessor — a link from a drink to the one it succeeds, capturing a product's lineage across reformulations or rebrands.

Localized drink data

Some drink data varies by region, so it is stored per country (and sometimes per language) rather than on the drink itself.

Localization — a drink's localized marketing name and optional descriptive text for one country + language combination (`DrinkLocalization`).

Composition — a drink's nutritional composition for a given country, with all values expressed **per 100 mL** (`DrinkComposition`).

Claim — a dietary or certification claim for a drink in a specific country (`DrinkClaim`), e.g. a dietary or regulatory label.

Reference data

Shared look-up values referenced across the catalogue:

Country · Language — the regional dimensions localized data is keyed on.

Can format — a standard can size/format.

Colour — a colour used to identify a can's design.

App version — a released version of the app; catalogue items record the version they were *introduced in*, which also drives the World Dex timeline.

People

User — an account holder. A user owns a Dex (their unlocked drinks) and is authenticated via Firebase — see Security.

Where to go next

- How these map to tables → Database
- How the layers use them → Back-end
- What the app does with them → Understanding Candex